CME 307 / MS&E 311 / OIT 676: Optimization

Conic optimization

From convex models to computable problems

Professor Udell

Management Science and Engineering
Stanford

October 22, 2026

Today

we know how to recognize a convex problem, characterize solutions, and build a dual

Today

we know how to recognize a convex problem, characterize solutions, and build a dual

today: **how should we hand a convex problem to a solver?**

- a **structured representation**: affine maps plus a few standard convex cones
- two important classes: second-order cone programs (SOCP) and semidefinite programs (SDP)
- **relaxations**: convex problems that bound hard ones
- conic duality: more practice building Lagrangian lower bounds

Outline

Robust models

Why expose conic structure?

Conic form

From a model to a solver

Semidefinite programs

Conic duality

A relaxation: MaxCut

An optimal production plan

choose amounts $x_1, x_2 \geq 0$ of two products, using one unit of capacity:

$$\begin{array}{ll} \text{maximize} & 3x_1 + 2x_2 \\ \text{subject to} & x_1 + x_2 \leq 1 \end{array}$$

An optimal production plan

choose amounts $x_1, x_2 \geq 0$ of two products, using one unit of capacity:

$$\begin{array}{ll} \text{maximize} & 3x_1 + 2x_2 \\ \text{subject to} & x_1 + x_2 \leq 1 \end{array}$$

Q: What is the optimal plan? How much capacity is left?

An optimal production plan

choose amounts $x_1, x_2 \geq 0$ of two products, using one unit of capacity:

$$\begin{array}{ll} \text{maximize} & 3x_1 + 2x_2 \\ \text{subject to} & x_1 + x_2 \leq 1 \end{array}$$

Q: What is the optimal plan? How much capacity is left?
A: $x = (1, 0)$; profit 3; no spare capacity.

An optimal production plan

choose amounts $x_1, x_2 \geq 0$ of two products, using one unit of capacity:

$$\begin{array}{ll} \text{maximize} & 3x_1 + 2x_2 \\ \text{subject to} & x_1 + x_2 \leq 1 \end{array}$$

Q: What is the optimal plan? How much capacity is left?
A: $x = (1, 0)$; profit 3; no spare capacity.

actual resource use is $a_1 x_1 + a_2 x_2$, and a is estimated

An optimal production plan

choose amounts $x_1, x_2 \geq 0$ of two products, using one unit of capacity:

$$\begin{array}{ll} \text{maximize} & 3x_1 + 2x_2 \\ \text{subject to} & x_1 + x_2 \leq 1 \end{array}$$

Q: What is the optimal plan? How much capacity is left?
A: $x = (1, 0)$; profit 3; no spare capacity.

actual resource use is $a_1x_1 + a_2x_2$, and a is estimated

Q: What happens if product 1 uses 10% more capacity than expected?

An optimal production plan

choose amounts $x_1, x_2 \geq 0$ of two products, using one unit of capacity:

$$\begin{array}{ll} \text{maximize} & 3x_1 + 2x_2 \\ \text{subject to} & x_1 + x_2 \leq 1 \end{array}$$

Q: What is the optimal plan? How much capacity is left?
A: $x = (1, 0)$; profit 3; no spare capacity.

actual resource use is $a_1x_1 + a_2x_2$, and a is estimated

Q: What happens if product 1 uses 10% more capacity than expected?
A: The plan is infeasible. Optimality depends on the model's data.

Decisions before uncertainty is revealed

model resource coefficients as

$$a = \bar{a} + Pu, \qquad u \in \mathcal{U} = \{u : \|u\|_2 \leq \rho\},$$

where $\bar{a} \in \mathbb{R}^n$, $P \in \mathbb{R}^{n \times k}$, and $\rho \geq 0$ are given

- P describes the directions and relative sizes of errors
- ρ controls how much uncertainty we protect against
- a lies in an **ellipsoid** centered at $\bar{a}$; the **nominal** model uses $a = \bar{a}$

Decisions before uncertainty is revealed

model resource coefficients as

$$a = \bar{a} + Pu, \qquad u \in \mathcal{U} = \{u : \|u\|_2 \leq \rho\},$$

where $\bar{a} \in \mathbb{R}^n$, $P \in \mathbb{R}^{n \times k}$, and $\rho \geq 0$ are given

- P describes the directions and relative sizes of errors
- ρ controls how much uncertainty we protect against
- a lies in an **ellipsoid** centered at $\bar{a}$; the **nominal** model uses $a = \bar{a}$

choose one x that works for **every** allowed error:

$$(\bar{a} + Pu)^T x \leq b \qquad \forall \|u\|_2 \leq \rho.$$

Decisions before uncertainty is revealed

model resource coefficients as

$$a = \bar{a} + Pu, \qquad u \in \mathcal{U} = \{u : \|u\|_2 \leq \rho\},$$

where $\bar{a} \in \mathbb{R}^n$, $P \in \mathbb{R}^{n \times k}$, and $\rho \geq 0$ are given

- P describes the directions and relative sizes of errors
- ρ controls how much uncertainty we protect against
- a lies in an **ellipsoid** centered at $\bar{a}$; the **nominal** model uses $a = \bar{a}$

choose one x that works for **every** allowed error:

$$(\bar{a} + Pu)^T x \leq b \qquad \forall \|u\|_2 \leq \rho.$$

Q: There are infinitely many constraints. Do we need to list them?

Decisions before uncertainty is revealed

model resource coefficients as

$$a = \bar{a} + Pu, \qquad u \in \mathcal{U} = \{u : \|u\|_2 \leq \rho\},$$

where $\bar{a} \in \mathbb{R}^n$, $P \in \mathbb{R}^{n \times k}$, and $\rho \geq 0$ are given

- P describes the directions and relative sizes of errors
- ρ controls how much uncertainty we protect against
- a lies in an **ellipsoid** centered at $\bar{a}$; the **nominal** model uses $a = \bar{a}$

choose one x that works for **every** allowed error:

$$(\bar{a} + Pu)^T x \leq b \qquad \forall \|u\|_2 \leq \rho.$$

Q: There are infinitely many constraints. Do we need to list them?
recall (LP duality applications): for a **polyhedral** uncertainty set, LP duality turned "$\forall u$" into finitely many linear constraints. Today: a Euclidean ball.

The worst error for a fixed decision

the robust constraint is equivalent to

$$\bar{a}^T x + \sup_{\|u\|_2 \leq \rho} u^T P^T x \leq b.$$

The worst error for a fixed decision

the robust constraint is equivalent to

$$\bar{a}^T x + \sup_{\|u\|_2 \le \rho} u^T P^T x \le b.$$

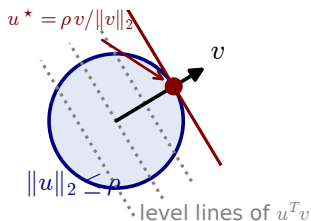
Q: Write $v = P^T x$. Which u maximizes $u^T v$ over $\|u\|_2 \le \rho$?

The worst error for a fixed decision

the robust constraint is equivalent to

$$\bar{a}^T x + \sup_{\|u\|_2 \le \rho} u^T P^T x \le b.$$

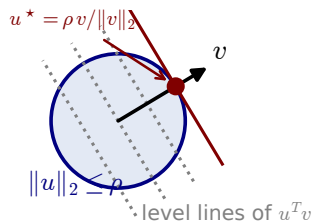
Q: Write $v = P^T x$. Which u maximizes $u^T v$ over $\|u\|_2 \le \rho$?
Cauchy–Schwarz gives $u^T v \le \|u\|_2 \|v\|_2 \le \rho \|v\|_2$, with equality at $u^\star = \rho v / \|v\|_2$, so
$\sup_{\|u\|_2 \le \rho} u^T v = \rho \|v\|_2$ (also when $v = 0$)



The worst error for a fixed decision

the robust constraint is equivalent to

$$\bar{a}^T x + \sup_{\|u\|_2 \le \rho} u^T P^T x \le b.$$



Q: Write $v = P^T x$. Which u maximizes $u^T v$ over $\|u\|_2 \le \rho$?
Cauchy–Schwarz gives $u^T v \le \|u\|_2 \|v\|_2 \le \rho \|v\|_2$,
with equality at $u^\star = \rho v/\|v\|_2$, so
$\sup_{\|u\|_2 \le \rho} u^T v = \rho \|v\|_2$ (also when $v = 0$)
hence the **exact robust counterpart** is

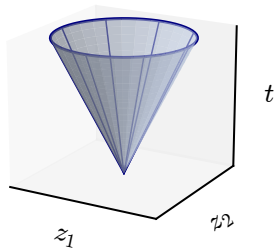
$$\boxed{\bar{a}^T x + \rho \|P^T x\|_2 \le b.}$$

this sup is the **support function** of $\mathcal{U}$, the Fenchel conjugate of its indicator

A second-order cone constraint

the **second-order cone** (SOC) is

$$\mathcal{Q}^{k+1} = \{(z, t) \in \mathbb{R}^k \times \mathbb{R} : \|z\|_2 \leq t\}.$$



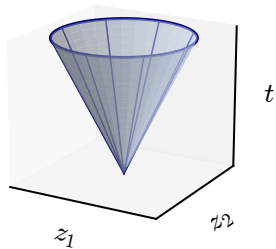
A second-order cone constraint

the **second-order cone** (SOC) is

$$\mathcal{Q}^{k+1} = \{(z, t) \in \mathbb{R}^k \times \mathbb{R} : \|z\|_2 \leq t\}.$$

our robust constraint has the form

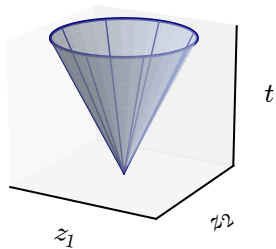
$$\|\rho P^T x\|_2 \leq b - \bar{a}^T x$$
$$\Updownarrow$$
$$(\rho P^T x,\ b - \bar{a}^T x) \in \mathcal{Q}^{k+1}$$



A second-order cone constraint

the **second-order cone** (SOC) is

$$\mathcal{Q}^{k+1} = \{(z, t) \in \mathbb{R}^k \times \mathbb{R} : \|z\|_2 \leq t\}.$$



our robust constraint has the form

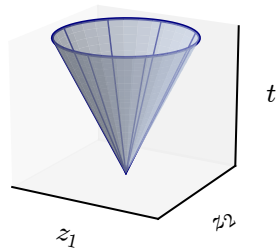
$$\|\rho P^T x\|_2 \leq b - \bar{a}^T x$$
$$\Updownarrow$$
$$(\rho P^T x,\ b - \bar{a}^T x) \in \mathcal{Q}^{k+1}$$

Q: What happens if the right-hand side is negative?

A second-order cone constraint

the **second-order cone** (SOC) is

$$\mathcal{Q}^{k+1} = \{(z, t) \in \mathbb{R}^k \times \mathbb{R} : \|z\|_2 \leq t\}.$$



our robust constraint has the form

$$\|\rho P^T x\|_2 \leq b - \bar{a}^T x$$
$$\Updownarrow$$
$$(\rho P^T x,\ b - \bar{a}^T x) \in \mathcal{Q}^{k+1}$$

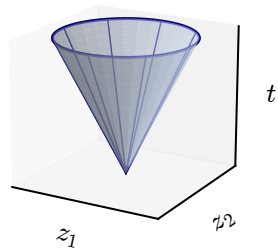
Q: What happens if the right-hand side is negative?

A: The constraint is infeasible: the norm is nonnegative.

A second-order cone constraint

the **second-order cone** (SOC) is

$$\mathcal{Q}^{k+1} = \{(z, t) \in \mathbb{R}^k \times \mathbb{R} : \|z\|_2 \leq t\}.$$



our robust constraint has the form

$$\|\rho P^T x\|_2 \leq b - \bar{a}^T x$$
$$\Updownarrow$$
$$(\rho P^T x,\ b - \bar{a}^T x) \in \mathcal{Q}^{k+1}$$

Q: What happens if the right-hand side is negative?

A: The constraint is infeasible: the norm is nonnegative.

infinitely many linear constraints become one cone constraint

a linear objective with affine and SOC constraints is a **second-order cone program** (SOCP)

Production with uncertain resource use

take $\bar{a} = (1, 1)$, $P = \mathbf{diag}(0.5, 0.1)$, and $\rho = 2$:

$$\begin{array}{ll} \text{maximize} & 3x_1 + 2x_2 \\ \text{subject to} & x_1 + x_2 + 2\sqrt{(0.5x_1)^2 + (0.1x_2)^2} \leq 1 \\ & x_1, x_2 \geq 0. \end{array}$$

Production with uncertain resource use

take $\bar{a} = (1, 1)$, $P = \mathbf{diag}(0.5, 0.1)$, and $\rho = 2$:

$$\begin{array}{ll} \text{maximize} & 3x_1 + 2x_2 \\ \text{subject to} & x_1 + x_2 + 2\sqrt{(0.5x_1)^2 + (0.1x_2)^2} \leq 1 \\ & x_1, x_2 \geq 0. \end{array}$$

$$x^\star \approx (0.1360,\ 0.6727), \qquad \text{profit} \approx 1.7534.$$

Production with uncertain resource use

take $\bar{a} = (1, 1)$, $P = \mathbf{diag}(0.5, 0.1)$, and $\rho = 2$:

$$\begin{array}{ll} \text{maximize} & 3x_1 + 2x_2 \\ \text{subject to} & x_1 + x_2 + 2\sqrt{(0.5x_1)^2 + (0.1x_2)^2} \leq 1 \\ & x_1, x_2 \geq 0. \end{array}$$

$$x^\star \approx (0.1360,\ 0.6727), \qquad \text{profit} \approx 1.7534.$$

the nominal plan $(1, 0)$ uses as much as **two** units of capacity

Production with uncertain resource use

take $\bar{a} = (1, 1)$, $P = \mathbf{diag}(0.5, 0.1)$, and $\rho = 2$:

$$\begin{array}{ll} \text{maximize} & 3x_1 + 2x_2 \\ \text{subject to} & x_1 + x_2 + 2\sqrt{(0.5x_1)^2 + (0.1x_2)^2} \leq 1 \\ & x_1, x_2 \geq 0. \end{array}$$

$$x^\star \approx (0.1360,\ 0.6727), \qquad \text{profit} \approx 1.7534.$$

the nominal plan $(1, 0)$ uses as much as **two** units of capacity

Q: Product 1 is more profitable. Why does the robust plan favor product 2?

Production with uncertain resource use

take $\bar{a} = (1, 1)$, $P = \mathbf{diag}(0.5, 0.1)$, and $\rho = 2$:

$$\begin{array}{ll} \text{maximize} & 3x_1 + 2x_2 \\ \text{subject to} & x_1 + x_2 + 2\sqrt{(0.5x_1)^2 + (0.1x_2)^2} \leq 1 \\ & x_1, x_2 \geq 0. \end{array}$$

$$x^\star \approx (0.1360,\ 0.6727), \qquad \text{profit} \approx 1.7534.$$

the nominal plan $(1, 0)$ uses as much as **two** units of capacity

Q: Product 1 is more profitable. Why does the robust plan favor product 2?
A: Product 1's resource use is five times as uncertain.

Exact reformulation, restriction, or relaxation?

three ways a new problem can relate to an old one:

- **exact reformulation**: same optimal value;
  solutions map back and forth

Exact reformulation, restriction, or relaxation?

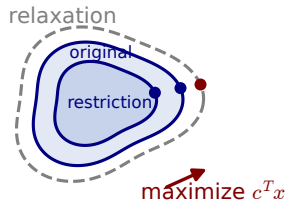
three ways a new problem can relate to an old one:

- **exact reformulation**: same optimal value; solutions map back and forth
- **restriction**: smaller feasible set; its solutions are feasible but may be conservative

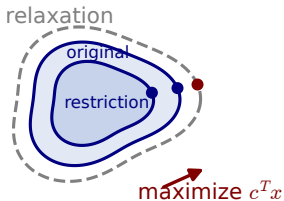
Exact reformulation, restriction, or relaxation?

three ways a new problem can relate to an old one:

- **exact reformulation**: same optimal value; solutions map back and forth
- **restriction**: smaller feasible set; its solutions are feasible but may be conservative
- **relaxation**: larger feasible set; its optimal value is a **bound**
  - an upper bound when we maximize, a lower bound when we minimize



Exact reformulation, restriction, or relaxation?

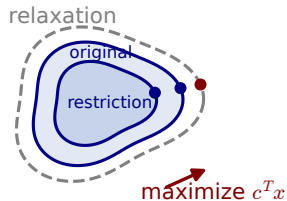


three ways a new problem can relate to an old one:

- **exact reformulation**: same optimal value; solutions map back and forth
- **restriction**: smaller feasible set; its solutions are feasible but may be conservative
- **relaxation**: larger feasible set; its optimal value is a **bound**
  - an upper bound when we maximize, a lower bound when we minimize

Q: Our SOC constraint: exact, restriction, or relaxation? Of which model?

Exact reformulation, restriction, or relaxation?



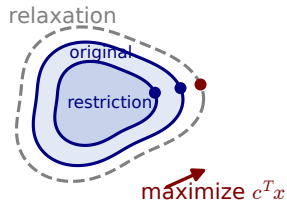
three ways a new problem can relate to an old one:

- **exact reformulation**: same optimal value; solutions map back and forth
- **restriction**: smaller feasible set; its solutions are feasible but may be conservative
- **relaxation**: larger feasible set; its optimal value is a **bound**
  - an upper bound when we maximize, a lower bound when we minimize

Q: Our SOC constraint: exact, restriction, or relaxation? Of which model?

A: Exact for the robust model; a restriction of the nominal model.

Exact reformulation, restriction, or relaxation?



three ways a new problem can relate to an old one:

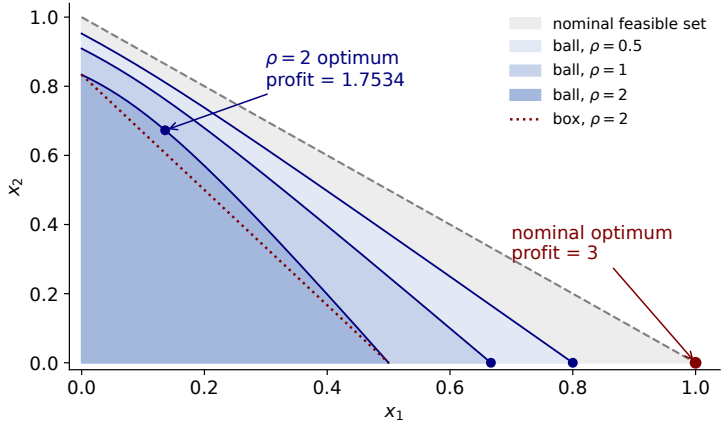
- **exact reformulation**: same optimal value; solutions map back and forth
- **restriction**: smaller feasible set; its solutions are feasible but may be conservative
- **relaxation**: larger feasible set; its optimal value is a **bound**
  - an upper bound when we maximize, a lower bound when we minimize

Q: Our SOC constraint: exact, restriction, or relaxation? Of which model?

A: Exact for the robust model; a restriction of the nominal model.

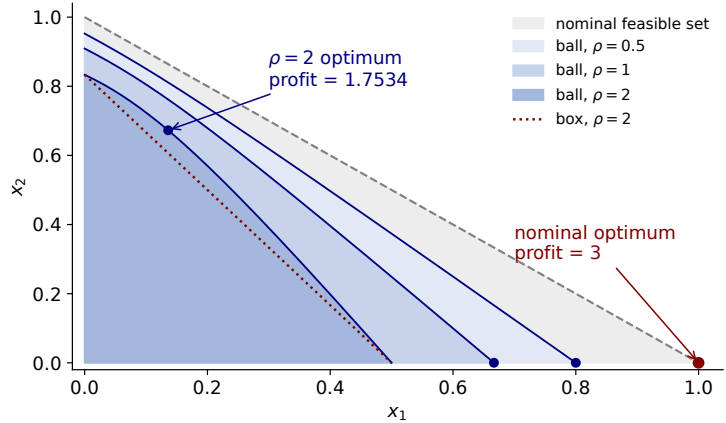
we will meet an important relaxation when we study MaxCut

The price of protection



Q: As ρ increases, can the optimal profit increase?

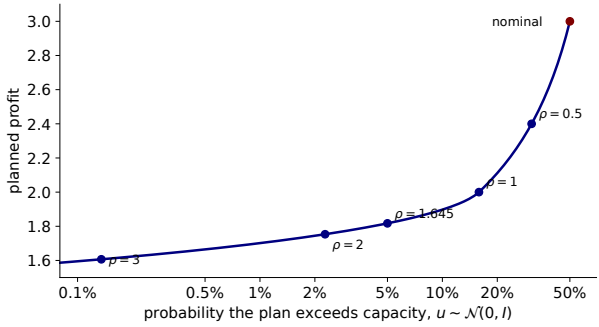
The price of protection



Q: As ρ increases, can the optimal profit increase?

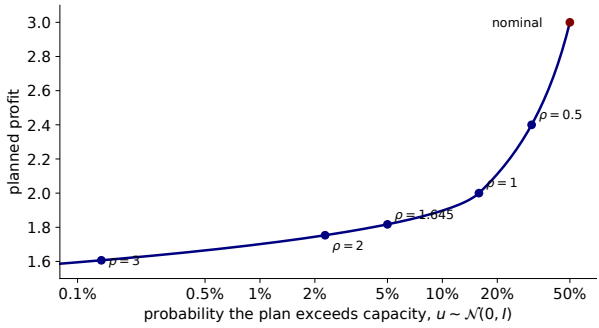
A: No: the feasible sets are nested. Larger ρ protects against more errors.

What protection buys



suppose the errors are random: $u \sim \mathcal{N}(0, I)$; Φ is the standard normal CDF

What protection buys

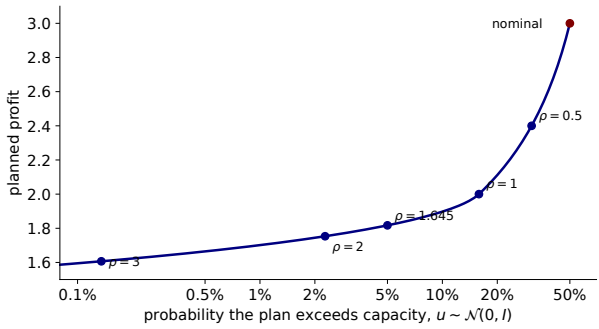


suppose the errors are random: $u \sim \mathcal{N}(0, I)$; Φ is the standard normal CDF

the ρ-robust plan exceeds capacity with probability exactly $1 - \Phi(\rho)$

- nominal: 50%
- $\rho = 1.645$: 5%, profit 1.82
- $\rho = 2$: 2.3%, profit 1.75

What protection buys



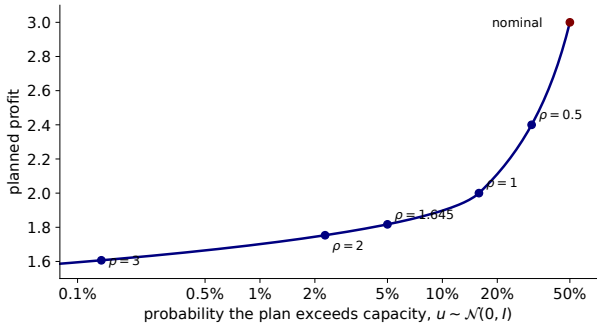
suppose the errors are random: $u \sim \mathcal{N}(0, I)$; Φ is the standard normal CDF

the ρ-robust plan exceeds capacity with probability exactly $1 - \Phi(\rho)$

- nominal: 50%
- $\rho = 1.645$: 5%, profit 1.82
- $\rho = 2$: 2.3%, profit 1.75

Q: Why exactly $1 - \Phi(\rho)$?

What protection buys



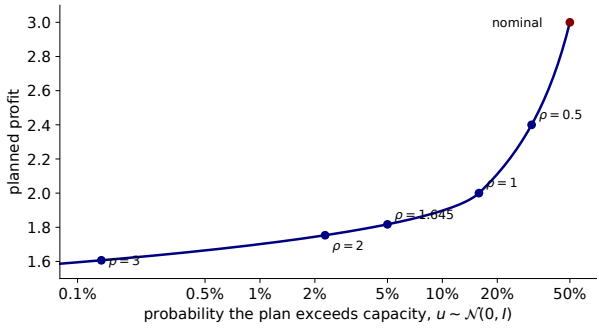
suppose the errors are random: $u \sim \mathcal{N}(0, I)$; Φ is the standard normal CDF

the ρ-robust plan exceeds capacity with probability exactly $1 - \Phi(\rho)$

- nominal: 50%
- $\rho = 1.645$: 5%, profit 1.82
- $\rho = 2$: 2.3%, profit 1.75

Q: Why exactly $1 - \Phi(\rho)$?
A: $u^T P^T x \sim \mathcal{N}(0, \|P^T x\|_2^2)$, and the robust optimum leaves slack exactly $\rho \|P^T x\|_2$.

What protection buys



suppose the errors are random: $u \sim \mathcal{N}(0, I)$; Φ is the standard normal CDF
the ρ-robust plan exceeds capacity with probability exactly $1 - \Phi(\rho)$

- nominal: 50%
- $\rho = 1.645$: 5%, profit 1.82
- $\rho = 2$: 2.3%, profit 1.75

Q: Why exactly $1 - \Phi(\rho)$?
A: $u^T P^T x \sim \mathcal{N}(0, \|P^T x\|_2^2)$, and the robust optimum leaves slack exactly $\rho \|P^T x\|_2$.

for Gaussian $a \sim \mathcal{N}(\bar{a}, PP^T)$ and $\eta \geq \frac{1}{2}$, the **chance constraint** $\mathbf{P}(a^T x \leq b) \geq \eta$ is the SOC constraint $\bar{a}^T x + \Phi^{-1}(\eta) \|P^T x\|_2 \leq b$

The uncertainty set is a modeling choice

allowed errors	worst extra resource use	robust counterpart
$\|u\|_2 \leq \rho$	$\rho\|P^T x\|_2$	SOCP
$\|u\|_\infty \leq \rho$	$\rho\|P^T x\|_1$ (choose $u_j = \rho\,\mathbf{sign}(v_j)$)	LP
$Du \leq d$	$\min_\eta\{d^T\eta : D^T\eta = P^T x,\ \eta \geq 0\}$ (LP dual)	LP

The uncertainty set is a modeling choice

allowed errors	worst extra resource use	robust counterpart
$\|u\|_2 \le \rho$	$\rho\|P^T x\|_2$	SOCP
$\|u\|_\infty \le \rho$	$\rho\|P^T x\|_1$ (choose $u_j = \rho\,\mathbf{sign}(v_j)$)	LP
$Du \le d$	$\min_\eta\{d^T\eta : D^T\eta = P^T x,\ \eta \ge 0\}$ (LP dual)	LP

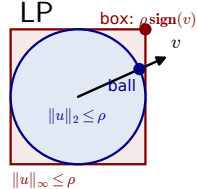
Q: For the same P and ρ, which model is more conservative?

The uncertainty set is a modeling choice

allowed errors	worst extra resource use	robust counterpart
$\|u\|_2 \le \rho$	$\rho\|P^T x\|_2$	SOCP
$\|u\|_\infty \le \rho$	$\rho\|P^T x\|_1$ (choose $u_j = \rho\,\mathbf{sign}(v_j)$)	LP
$Du \le d$	$\min_\eta\{d^T\eta : D^T\eta = P^T x,\ \eta \ge 0\}$ (LP dual)	LP

Q: For the same P and ρ, which model is more conservative?

A: The box contains the ball: it protects against more errors.



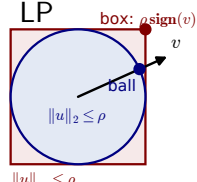
The uncertainty set is a modeling choice

allowed errors	worst extra resource use	robust counterpart
$\|u\|_2 \le \rho$	$\rho\|P^T x\|_2$	SOCP
$\|u\|_\infty \le \rho$	$\rho\|P^T x\|_1$ (choose $u_j = \rho\,\mathbf{sign}(v_j)$)	LP
$Du \le d$	$\min_\eta\{d^T\eta : D^T\eta = P^T x,\ \eta \ge 0\}$ (LP dual)	LP

Q: For the same P and ρ, which model is more conservative?

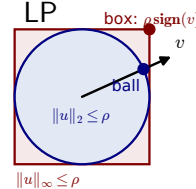
A: The box contains the ball: it protects against more errors.

- choose P, ρ from domain knowledge and data; validate out of sample
- protection holds only for errors **inside** the chosen set



The uncertainty set is a modeling choice

allowed errors	worst extra resource use	robust counterpart
$\|u\|_2 \le \rho$	$\rho\|P^T x\|_2$	SOCP
$\|u\|_\infty \le \rho$	$\rho\|P^T x\|_1$ (choose $u_j = \rho\,\mathbf{sign}(v_j)$)	LP
$Du \le d$	$\min_\eta\{d^T\eta : D^T\eta = P^T x,\ \eta \ge 0\}$ (LP dual)	LP



Q: For the same P and ρ, which model is more conservative?

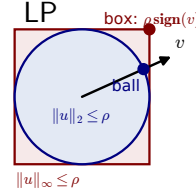
A: The box contains the ball: it protects against more errors.

- choose P, ρ from domain knowledge and data; validate out of sample
- protection holds only for errors **inside** the chosen set

Q: Does $\rho = 2$ imply a 95% probability guarantee?

The uncertainty set is a modeling choice

allowed errors	worst extra resource use	robust counterpart
$\|u\|_2 \le \rho$	$\rho\|P^T x\|_2$	SOCP
$\|u\|_\infty \le \rho$	$\rho\|P^T x\|_1$ (choose $u_j = \rho\,\mathbf{sign}(v_j)$)	LP
$Du \le d$	$\min_\eta\{d^T\eta : D^T\eta = P^T x,\ \eta \ge 0\}$ (LP dual)	LP



Q: For the same P and ρ, which model is more conservative?
A: The box contains the ball: it protects against more errors.

- choose P, ρ from domain knowledge and data; validate out of sample
- protection holds only for errors **inside** the chosen set

Q: Does $\rho = 2$ imply a 95% probability guarantee?
A: Only under a distributional model (e.g., Gaussian); otherwise calibrate.

Outline

Robust models

Why expose conic structure?

Conic form

From a model to a solver

Semidefinite programs

Conic duality

A relaxation: MaxCut

How do we hand this constraint to a solver?

$$\bar{a}^T x + \rho \| P^T x \|_2 \leq b$$

How do we hand this constraint to a solver?

$$\bar{a}^T x + \rho \|P^T x\|_2 \leq b$$

- **as written**: not differentiable where $P^T x = 0$

How do we hand this constraint to a solver?

$$\bar{a}^T x + \rho \|P^T x\|_2 \leq b$$

- **as written**: not differentiable where $P^T x = 0$
- **squared**: $\rho^2 \|P^T x\|_2^2 \leq (b - \bar{a}^T x)^2$ with $b - \bar{a}^T x \geq 0$
  - smooth, but not obviously convex, and gradient is misleading at the tip ($P^T x = 0$, $b = \bar{a}^T x$)

How do we hand this constraint to a solver?

$$\bar{a}^T x + \rho \|P^T x\|_2 \leq b$$

- **as written**: not differentiable where $P^T x = 0$
- **squared**: $\rho^2 \|P^T x\|_2^2 \leq (b - \bar{a}^T x)^2$ with $b - \bar{a}^T x \geq 0$
  - smooth, but not obviously convex, and gradient is misleading at the tip ($P^T x = 0,\ b = \bar{a}^T x$)
- **as a cone**: $(\rho P^T x,\ b - \bar{a}^T x) \in \mathcal{Q}^{k+1}$

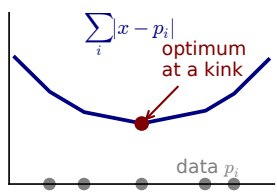
How do we hand this constraint to a solver?

$$\bar{a}^T x + \rho \|P^T x\|_2 \leq b$$

- **as written**: not differentiable where $P^T x = 0$
- **squared**: $\rho^2 \|P^T x\|_2^2 \leq (b - \bar{a}^T x)^2$ with $b - \bar{a}^T x \geq 0$
  - smooth, but not obviously convex, and gradient is misleading at the tip ($P^T x = 0$, $b = \bar{a}^T x$)
- **as a cone**: $(\rho P^T x,\ b - \bar{a}^T x) \in \mathcal{Q}^{k+1}$

nonsmooth convex functions are common, not exotic:

- ℓ_1 penalties, max of affine functions, $\lambda_{\max}$ (largest eigenvalue) of a symmetric matrix
- facility location: minimize $\sum_i \|x - p_i\|_2$; often optimal at a kink $x = p_i$



Two solver interfaces

Derivative oracles: IPOPT

$$f(x),\ g(x),\ \nabla f(x),\ J_g(x),\ \nabla^2_{xx}\mathcal{L}$$

- provide function values and derivatives, often by automatic differentiation
- assumes twice-differentiable functions
- flexible smooth models, including nonconvex problems

Conic structure: Clarabel, MOSEK

$$A,\ b,\ c,\quad \text{standard cone types}$$

- provide affine maps and cone constraints
- the solver has specialized routines for every cone type
- convexity is built into the input form

Two solver interfaces

Derivative oracles: IPOPT

$f(x),\ g(x),\ \nabla f(x),\ J_g(x),\ \nabla^2_{xx}\mathcal{L}$

- provide function values and derivatives, often by automatic differentiation
- assumes twice-differentiable functions
- flexible smooth models, including nonconvex problems

Conic structure: Clarabel, MOSEK

$A,\ b,\ c,$ standard cone types

- provide affine maps and cone constraints
- the solver has specialized routines for every cone type
- convexity is built into the input form

IPOPT and Clarabel both use **interior-point methods** (after the midterm)

Why expose conic structure?

- **nonsmooth functions, exactly**: norms, absolute values, maxima become affine maps plus cones, with no derivative of a kink

Why expose conic structure?

- **nonsmooth functions, exactly**: norms, absolute values, maxima become affine maps plus cones, with no derivative of a kink
- **global answers with certificates** (proofs you can check): a primal–dual pair that proves optimality, or a proof of infeasibility

Why expose conic structure?

- **nonsmooth functions, exactly**: norms, absolute values, maxima become affine maps plus cones, with no derivative of a kink
- **global answers with certificates** (proofs you can check): a primal–dual pair that proves optimality, or a proof of infeasibility
- **constraints with no good smooth form**: $X \succeq 0$ (X positive semidefinite) and $\lambda_{\max}$ (eigenvalues are not differentiable where they coincide)

Why expose conic structure?

- **nonsmooth functions, exactly**: norms, absolute values, maxima become affine maps plus cones, with no derivative of a kink
- **global answers with certificates** (proofs you can check): a primal–dual pair that proves optimality, or a proof of infeasibility
- **constraints with no good smooth form**: $X \succeq 0$ (X positive semidefinite) and $\lambda_{\max}$ (eigenvalues are not differentiable where they coincide)
- **predictable work**: interior-point iteration counts barely grow with problem size

Why expose conic structure?

- **nonsmooth functions, exactly**: norms, absolute values, maxima become affine maps plus cones, with no derivative of a kink
- **global answers with certificates** (proofs you can check): a primal–dual pair that proves optimality, or a proof of infeasibility
- **constraints with no good smooth form**: $X \succeq 0$ (X positive semidefinite) and $\lambda_{\max}$ (eigenvalues are not differentiable where they coincide)
- **predictable work**: interior-point iteration counts barely grow with problem size

the tradeoff:

- extra variables and constraints can enlarge the numerical work
- a compact smooth model can also be very effective

Why expose conic structure?

- **nonsmooth functions, exactly**: norms, absolute values, maxima become affine maps plus cones, with no derivative of a kink
- **global answers with certificates** (proofs you can check): a primal–dual pair that proves optimality, or a proof of infeasibility
- **constraints with no good smooth form**: $X \succeq 0$ (X positive semidefinite) and $\lambda_{\max}$ (eigenvalues are not differentiable where they coincide)
- **predictable work**: interior-point iteration counts barely grow with problem size

the tradeoff:

- extra variables and constraints can enlarge the numerical work
- a compact smooth model can also be very effective

Q: If two formulations are mathematically equivalent, must they take the same time?

When should you use a derivative-oracle solver?

conic solver (Clarabel, MOSEK, via CVXPY)

- convex: norms, quadratics, logs, matrix inequalities
- nonsmooth terms: ℓ_1, max, $\lambda_{\max}$
- you need certificates or infeasibility detection

derivative oracle (IPOPT, KNITRO, SNOPT)

- smooth **nonconvex** models: power flow, chemical processes, ODE dynamics
- functions given by simulation code or a neural network
- a compact smooth model the conic form would enlarge

When should you use a derivative-oracle solver?

conic solver (Clarabel, MOSEK, via CVXPY)

- convex: norms, quadratics, logs, matrix inequalities
- nonsmooth terms: ℓ_1, max, $\lambda_{\max}$
- you need certificates or infeasibility detection

derivative oracle (IPOPT, KNITRO, SNOPT)

- smooth **nonconvex** models: power flow, chemical processes, ODE dynamics
- functions given by simulation code or a neural network
- a compact smooth model the conic form would enlarge

very large problems with cheap gradients: first-order methods (after the midterm)

When should you use a derivative-oracle solver?

conic solver (Clarabel, MOSEK, via CVXPY)

- convex: norms, quadratics, logs, matrix inequalities
- nonsmooth terms: ℓ_1, max, $\lambda_{\max}$
- you need certificates or infeasibility detection

derivative oracle (IPOPT, KNITRO, SNOPT)

- smooth **nonconvex** models: power flow, chemical processes, ODE dynamics
- functions given by simulation code or a neural network
- a compact smooth model the conic form would enlarge

very large problems with cheap gradients: first-order methods (after the midterm)

Q: IPOPT finds a KKT point of a convex problem. Is it globally optimal?

When should you use a derivative-oracle solver?

conic solver (Clarabel, MOSEK, via CVXPY)

- convex: norms, quadratics, logs, matrix inequalities
- nonsmooth terms: ℓ_1, max, $\lambda_{\max}$
- you need certificates or infeasibility detection

derivative oracle (IPOPT, KNITRO, SNOPT)

- smooth **nonconvex** models: power flow, chemical processes, ODE dynamics
- functions given by simulation code or a neural network
- a compact smooth model the conic form would enlarge

very large problems with cheap gradients: first-order methods (after the midterm)

Q: IPOPT finds a KKT point of a convex problem. Is it globally optimal?
A: Yes. But IPOPT cannot certify infeasibility, and kinks break its assumptions.

Outline

Robust models

Why expose conic structure?

Conic form

From a model to a solver

Semidefinite programs

Conic duality

A relaxation: MaxCut

Convex cones

Definition (Convex cone)

A nonempty set K is a convex cone if

$$x, y \in K, \quad \alpha, \beta \geq 0 \quad \Longrightarrow \quad \alpha x + \beta y \in K.$$

Convex cones

Definition (Convex cone)

A nonempty set K is a convex cone if

$$x, y \in K, \quad \alpha, \beta \geq 0 \quad \Longrightarrow \quad \alpha x + \beta y \in K.$$

cone	membership	models
$\{0\}$	$z = 0$	equalities
$\mathbb{R}^m_+$	$z \geq 0$	linear inequalities
$\mathcal{Q}^{m+1}$	$\|z\|_2 \leq t$	norm bounds, quadratics
$\mathbf{S}^m_+$ (PSD)	$Z = Z^T \succeq 0$	$z^T Z z \geq 0$ for every z
$K_{\exp}$	$ye^{x/y} \leq z,\ y > 0$ + closure	exp, log, entropy, logistic loss

the **power cone** $\{(x, y, z) : x^\alpha y^{1-\alpha} \geq |z|,\ x, y \geq 0\}$, $\alpha \in (0, 1)$, models $|x|^p$ and p-norms

Convex cones

Definition (Convex cone)

A nonempty set K is a convex cone if

$$x, y \in K, \quad \alpha, \beta \geq 0 \quad \Longrightarrow \quad \alpha x + \beta y \in K.$$

cone	membership	models
$\{0\}$	$z = 0$	equalities
$\mathbb{R}^m_+$	$z \geq 0$	linear inequalities
$\mathcal{Q}^{m+1}$	$\|z\|_2 \leq t$	norm bounds, quadratics
$\mathbf{S}^m_+$ (PSD)	$Z = Z^T \succeq 0$	$z^T Z z \geq 0$ for every z
$K_{\exp}$	$ye^{x/y} \leq z,\ y > 0$ + closure	exp, log, entropy, logistic loss

the **power cone** $\{(x, y, z) : x^\alpha y^{1-\alpha} \geq |z|,\ x, y \geq 0\}$, $\alpha \in (0, 1)$, models $|x|^p$ and p-norms

Q: Why is the second-order cone convex?

Conic form

a **conic program** has the form

$$\begin{array}{ll} \text{minimize} & c^T x \\ \text{subject to} & b - Ax \in K. \end{array}$$

here K is a closed convex cone; matrix-valued constraints use linear maps

Conic form

a **conic program** has the form

$$\begin{array}{ll} \text{minimize} & c^T x \\ \text{subject to} & b - Ax \in K. \end{array}$$

here K is a closed convex cone; matrix-valued constraints use linear maps

$$z_1 \in K_1,\ z_2 \in K_2 \quad \Longleftrightarrow \quad (z_1, z_2) \in K_1 \times K_2.$$

- use **products** to combine different constraint types
- affine constraints plus nonnegative cones give an LP
- adding second-order cones gives an SOCP; adding PSD cones gives a **semidefinite program** (SDP)

Conic form

a **conic program** has the form

$$\begin{array}{ll} \text{minimize} & c^T x \\ \text{subject to} & b - Ax \in K. \end{array}$$

here K is a closed convex cone; matrix-valued constraints use linear maps

$$z_1 \in K_1,\ z_2 \in K_2 \quad \iff \quad (z_1, z_2) \in K_1 \times K_2.$$

- use **products** to combine different constraint types
- affine constraints plus nonnegative cones give an LP
- adding second-order cones gives an SOCP; adding PSD cones gives a **semidefinite program** (SDP)

Q: Does writing an arbitrary cone K make a problem easy?

Conic form

a **conic program** has the form

$$\begin{array}{ll} \text{minimize} & c^T x \\ \text{subject to} & b - Ax \in K. \end{array}$$

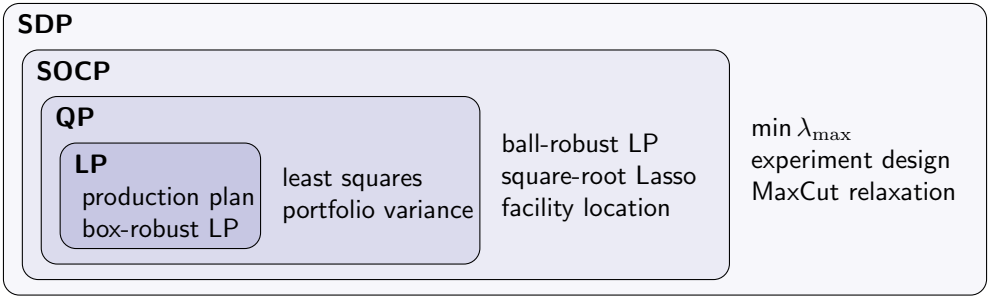
here K is a closed convex cone; matrix-valued constraints use linear maps

$$z_1 \in K_1,\ z_2 \in K_2 \quad \Longleftrightarrow \quad (z_1, z_2) \in K_1 \times K_2.$$

- use **products** to combine different constraint types
- affine constraints plus nonnegative cones give an LP
- adding second-order cones gives an SOCP; adding PSD cones gives a **semidefinite program** (SDP)

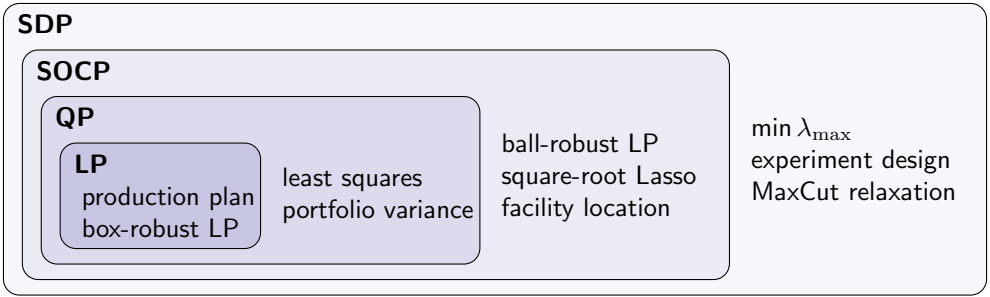
Q: Does writing an arbitrary cone K make a problem easy?
A: No. We need cones with usable algorithms and a manageable representation.

A hierarchy of conic programs



- QP: convex quadratic objective, affine constraints
- each inclusion is an exact reformulation (appendix)
- the exponential cone is **outside** this picture: no SDP represents exp or log exactly (they are not semialgebraic); solvers mix all these cones in one problem

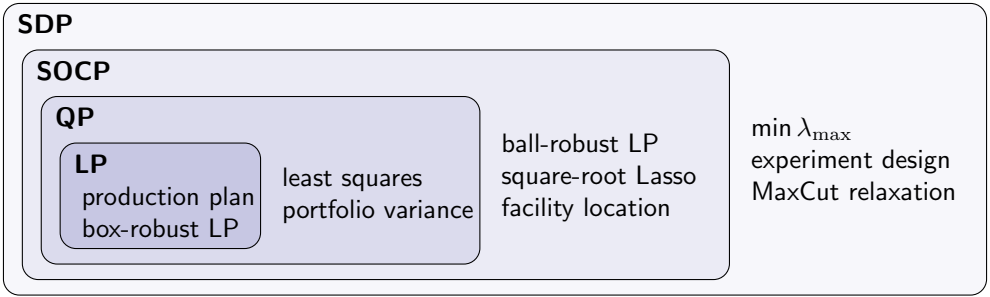
A hierarchy of conic programs



- QP: convex quadratic objective, affine constraints
- each inclusion is an exact reformulation (appendix)
- the exponential cone is **outside** this picture: no SDP represents exp or log exactly (they are not semialgebraic); solvers mix all these cones in one problem

Q: SDP contains everything here. Why not always formulate an SDP?

A hierarchy of conic programs



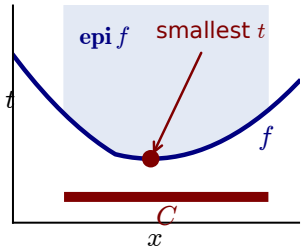
- QP: convex quadratic objective, affine constraints
- each inclusion is an exact reformulation (appendix)
- the exponential cone is **outside** this picture: no SDP represents exp or log exactly (they are not semialgebraic); solvers mix all these cones in one problem

Q: SDP contains everything here. Why not always formulate an SDP?
A: More general cones cost more per iteration; we will see the numbers.

The epigraph trick

recall $\mathbf{epi}\, f = \{(x,t) : f(x) \leq t\}$:

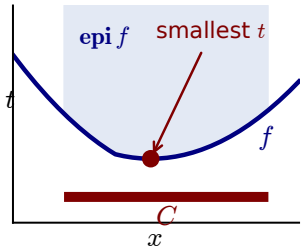
$$\min_{x \in C} f(x) \quad \Longleftrightarrow \quad \begin{array}{ll} \text{minimize}_{x,t} & t \\ \text{subject to} & f(x) \leq t, \quad x \in C. \end{array}$$



The epigraph trick

recall **epi** $f = \{(x,t) : f(x) \leq t\}$:

$$\min_{x \in C} f(x) \quad \Longleftrightarrow \quad \begin{array}{ll} \text{minimize}_{x,t} & t \\ \text{subject to} & f(x) \leq t, \quad x \in C. \end{array}$$



a **conic representation** expresses the epigraph using affine maps, standard cones, and possibly extra variables

The direction of the inequality matters

$$\|x\|_1 \leq 1 \quad \Longleftrightarrow \quad \exists s: \; -s \leq x \leq s, \; \mathbf{1}^T s \leq 1.$$

The direction of the inequality matters

$$\|x\|_1 \leq 1 \quad \Longleftrightarrow \quad \exists s: \; -s \leq x \leq s, \; \mathbf{1}^T s \leq 1.$$

Q: What if we replace both ≤ 1 inequalities by ≥ 1?

The direction of the inequality matters

$$\|x\|_1 \leq 1 \quad \Longleftrightarrow \quad \exists s: \ -s \leq x \leq s, \ \mathbf{1}^T s \leq 1.$$

Q: What if we replace both ≤ 1 inequalities by ≥ 1?
the proposed formulation is

$$\exists s: \ -s \leq x \leq s, \ \mathbf{1}^T s \geq 1.$$

The direction of the inequality matters

$$\|x\|_1 \leq 1 \quad \Longleftrightarrow \quad \exists s: \ -s \leq x \leq s, \ \mathbf{1}^T s \leq 1.$$

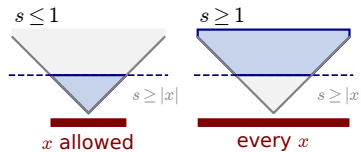
Q: What if we replace both ≤ 1 inequalities by ≥ 1?
the proposed formulation is

$$\exists s: \ -s \leq x \leq s, \ \mathbf{1}^T s \geq 1.$$

Q: Is $x = 0$ feasible? Can you find an s?

The direction of the inequality matters

$$\|x\|_1 \leq 1 \quad \Longleftrightarrow \quad \exists s: \ -s \leq x \leq s, \ \mathbf{1}^T s \leq 1.$$



Q: What if we replace both ≤ 1 inequalities by ≥ 1?
the proposed formulation is

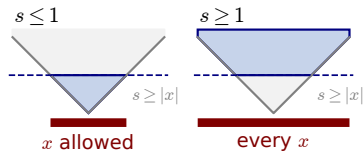
$$\exists s: \ -s \leq x \leq s, \ \mathbf{1}^T s \geq 1.$$

Q: Is $x = 0$ feasible? Can you find an s?
A: Yes: choose $s \geq 0$ with $\mathbf{1}^T s = 1$. But $\|0\|_1 \not\geq 1$.

picture: $n = 1$; shade the lifted set, then project onto x

The direction of the inequality matters

$$\|x\|_1 \leq 1 \quad \Longleftrightarrow \quad \exists s: \ -s \leq x \leq s, \ \mathbf{1}^T s \leq 1.$$



Q: What if we replace both ≤ 1 inequalities by ≥ 1?
the proposed formulation is

$$\exists s: \ -s \leq x \leq s, \ \mathbf{1}^T s \geq 1.$$

Q: Is $x = 0$ feasible? Can you find an s?
A: Yes: choose $s \geq 0$ with $\mathbf{1}^T s = 1$. But $\|0\|_1 \not\geq 1$.

picture: $n = 1$; shade the lifted set, then project onto x

bounding a function from above does not enforce a lower bound on it

Exercise: a complete conic model

square-root Lasso trades off residual size and coefficient sparsity:

$$\underset{x \in \mathbb{R}^n}{\text{minimize}} \quad \|Ax - b\|_2 + \gamma \|x\|_1, \qquad \gamma > 0,$$

where $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$

Q: Transform this problem to conic form.

- what extra variables do you need?
- which constraints use which cones?
- why is the reformulation exact?

Square-root Lasso in conic form

$$
\begin{array}{lllll}
\text{minimize}_{x,s,r,t} & t + \gamma \mathbf{1}^T s & & & \\
\text{subject to} & s - x \geq 0, & s + x \geq 0 & \mathbb{R}^n_+ \times \mathbb{R}^n_+ \\
& r = Ax - b & & \{0\}^m \\
& \|r\|_2 \leq t & & \mathcal{Q}^{m+1}.
\end{array}
$$

Square-root Lasso in conic form

$$
\begin{array}{lllll}
\text{minimize}_{x,s,r,t} & t + \gamma \mathbf{1}^T s & & & \\
\text{subject to} & s - x \geq 0, & s + x \geq 0 & \mathbb{R}^n_+ \times \mathbb{R}^n_+ \\
& r = Ax - b & & \{0\}^m \\
& \|r\|_2 \leq t & & \mathcal{Q}^{m+1}.
\end{array}
$$

- every x lifts via $s = |x|$, $r = Ax - b$, $t = \|r\|_2$ with the same objective
- every lifted feasible point has

$$\|Ax - b\|_2 + \gamma \|x\|_1 \leq t + \gamma \mathbf{1}^T s$$

Square-root Lasso in conic form

$$
\begin{array}{lllll}
\text{minimize}_{x,s,r,t} & t + \gamma \mathbf{1}^T s & & & \\
\text{subject to} & s - x \geq 0, & s + x \geq 0 & \mathbb{R}^n_+ \times \mathbb{R}^n_+ \\
& r = Ax - b & & \{0\}^m \\
& \|r\|_2 \leq t & & \mathcal{Q}^{m+1}.
\end{array}
$$

- every x lifts via $s = |x|$, $r = Ax - b$, $t = \|r\|_2$ with the same objective
- every lifted feasible point has

$$\|Ax - b\|_2 + \gamma \|x\|_1 \leq t + \gamma \mathbf{1}^T s$$

Q: What makes the two inequalities tight at an optimum?

Square-root Lasso in conic form

$$
\begin{array}{lll}
\text{minimize}_{x,s,r,t} & t + \gamma \mathbf{1}^T s & \\
\text{subject to} & s - x \geq 0, \quad s + x \geq 0 & \mathbb{R}^n_+ \times \mathbb{R}^n_+ \\
& r = Ax - b & \{0\}^m \\
& \|r\|_2 \leq t & \mathcal{Q}^{m+1}.
\end{array}
$$

- every x lifts via $s = |x|$, $r = Ax - b$, $t = \|r\|_2$ with the same objective
- every lifted feasible point has

$$\|Ax - b\|_2 + \gamma \|x\|_1 \leq t + \gamma \mathbf{1}^T s$$

Q: What makes the two inequalities tight at an optimum?
A: Both upper bounds have positive coefficients in a minimized objective.

What did lifting change?

the original problem has n variables; our formulation has $2n + m + 1$

What did lifting change?

the original problem has n variables; our formulation has $2n + m + 1$

$$\begin{pmatrix} s - x \\ s + x \\ r - Ax + b \\ r \\ t \end{pmatrix} \in \mathbb{R}^n_+ \times \mathbb{R}^n_+ \times \{0\}^m \times \mathcal{Q}^{m+1}.$$

What did lifting change?

the original problem has n variables; our formulation has $2n + m + 1$

$$\begin{pmatrix} s - x \\ s + x \\ r - Ax + b \\ r \\ t \end{pmatrix} \in \mathbb{R}^n_+ \times \mathbb{R}^n_+ \times \{0\}^m \times \mathcal{Q}^{m+1}.$$

the objective and all expressions inside the cone constraints are affine

What did lifting change?

the original problem has n variables; our formulation has $2n + m + 1$

$$\begin{pmatrix} s - x \\ s + x \\ r - Ax + b \\ r \\ t \end{pmatrix} \in \mathbb{R}^n_+ \times \mathbb{R}^n_+ \times \{0\}^m \times \mathcal{Q}^{m+1}.$$

the objective and all expressions inside the cone constraints are affine

Q: Can we eliminate r? Does that change the optimal value?

What did lifting change?

the original problem has n variables; our formulation has $2n + m + 1$

$$\begin{pmatrix} s - x \\ s + x \\ r - Ax + b \\ r \\ t \end{pmatrix} \in \mathbb{R}^n_+ \times \mathbb{R}^n_+ \times \{0\}^m \times \mathcal{Q}^{m+1}.$$

the objective and all expressions inside the cone constraints are affine

Q: Can we eliminate r? Does that change the optimal value?
A: Yes: use $(Ax - b, t) \in \mathcal{Q}^{m+1}$. The value is unchanged.

What did lifting change?

the original problem has n variables; our formulation has $2n + m + 1$

$$\begin{pmatrix} s - x \\ s + x \\ r - Ax + b \\ r \\ t \end{pmatrix} \in \mathbb{R}^n_+ \times \mathbb{R}^n_+ \times \{0\}^m \times \mathcal{Q}^{m+1}.$$

the objective and all expressions inside the cone constraints are affine

Q: Can we eliminate r? Does that change the optimal value?
A: Yes: use $(Ax - b, t) \in \mathcal{Q}^{m+1}$. The value is unchanged.
equivalent formulations can have different sizes and sparsity patterns

Outline

Robust models

Why expose conic structure?

Conic form

From a model to a solver

Semidefinite programs

Conic duality

A relaxation: MaxCut

Square-root Lasso: what the solvers receive

both solvers minimize $t + \gamma \mathbf{1}^T s$ over the same variables (x, s, r, t), with $-s \leq x \leq s$ and $r = Ax - b$

Square-root Lasso: what the solvers receive

both solvers minimize $t + \gamma \mathbf{1}^T s$ over the same variables (x, s, r, t), with $-s \leq x \leq s$ and $r = Ax - b$

IPOPT	Clarabel
$r^T r \leq t^2, \quad t \geq 0$	$(r, t) \in \mathcal{Q}^{m+1}$
smooth scalar constraint and its derivatives	a standard cone constraint

Square-root Lasso: what the solvers receive

both solvers minimize $t + \gamma \mathbf{1}^T s$ over the same variables (x, s, r, t), with $-s \leq x \leq s$ and $r = Ax - b$

IPOPT	Clarabel
$r^T r \leq t^2, \quad t \geq 0$	$(r, t) \in \mathcal{Q}^{m+1}$
smooth scalar constraint and its derivatives	a standard cone constraint

Q: Why do we need $t \geq 0$ in the scalar formulation?

Square-root Lasso: what the solvers receive

both solvers minimize $t + \gamma \mathbf{1}^T s$ over the same variables (x, s, r, t), with $-s \leq x \leq s$ and $r = Ax - b$

IPOPT	Clarabel
$r^T r \leq t^2, \quad t \geq 0$	$(r, t) \in \mathcal{Q}^{m+1}$
smooth scalar constraint and its derivatives	a standard cone constraint

Q: Why do we need $t \geq 0$ in the scalar formulation?
A: Otherwise negative t could satisfy the squared inequality.

Square-root Lasso: what the solvers receive

both solvers minimize $t + \gamma \mathbf{1}^T s$ over the same variables (x, s, r, t), with $-s \leq x \leq s$ and $r = Ax - b$

IPOPT	Clarabel
$r^T r \leq t^2, \quad t \geq 0$	$(r, t) \in \mathcal{Q}^{m+1}$
smooth scalar constraint and its derivatives	a standard cone constraint

Q: Why do we need $t \geq 0$ in the scalar formulation?
A: Otherwise negative t could satisfy the squared inequality.
both describe the same convex feasible set; the scalar function $r^T r - t^2$ itself is not convex

Square-root Lasso: what the solvers receive

both solvers minimize $t + \gamma \mathbf{1}^T s$ over the same variables (x, s, r, t), with $-s \leq x \leq s$ and $r = Ax - b$

IPOPT	Clarabel
$r^T r \leq t^2, \quad t \geq 0$	$(r, t) \in \mathcal{Q}^{m+1}$
smooth scalar constraint and its derivatives	a standard cone constraint

Q: Why do we need $t \geq 0$ in the scalar formulation?
A: Otherwise negative t could satisfy the squared inequality.
both describe the same convex feasible set; the scalar function $r^T r - t^2$ itself is not convex

notice: to give IPOPT a smooth problem, **we did the conic lifting ourselves**

Square-root Lasso: measured solve times

	solve only (ms)		setup + solve (ms)		iterations	
(m, n)	IPOPT	Clarabel	IPOPT	Clarabel	IPOPT	Clarabel
$(100, 50)$	40.9	6.4	50.8	12.2	24	13
$(300, 150)$	168.1	83.6	208.6	103.3	25	12
$(600, 300)$	873.7	610.6	1017.3	677.7	26	15

dense Gaussian $A \in \mathbb{R}^{m \times n}$, sparse signal plus noise; median of three runs on a laptop

- IPOPT via CasADi with exact derivatives; native Clarabel interface
- one thread; setup includes model construction and solver initialization
- all solves pass the same 10^{-6} relative feasibility and primal–dual gap checks

Square-root Lasso: measured solve times

	solve only (ms)		setup + solve (ms)		iterations	
(m, n)	IPOPT	Clarabel	IPOPT	Clarabel	IPOPT	Clarabel
$(100, 50)$	40.9	6.4	50.8	12.2	24	13
$(300, 150)$	168.1	83.6	208.6	103.3	25	12
$(600, 300)$	873.7	610.6	1017.3	677.7	26	15

dense Gaussian $A \in \mathbb{R}^{m \times n}$, sparse signal plus noise; median of three runs on a laptop

- IPOPT via CasADi with exact derivatives; native Clarabel interface
- one thread; setup includes model construction and solver initialization
- all solves pass the same 10^{-6} relative feasibility and primal–dual gap checks

Clarabel is faster here, with iteration counts nearly flat in problem size; the advantage depends on the problem and formulation

What if we skip the lifting?

hand IPOPT the original objective, with exact automatic derivatives:

$$f(x) = \|Ax - b\|_2 + \gamma\|x\|_1$$

What if we skip the lifting?

hand IPOPT the original objective, with exact automatic derivatives:

$$f(x) = \|Ax - b\|_2 + \gamma\|x\|_1$$

same 9 instances, same tolerance 10^{-9}, same 500-iteration limit:

- IPOPT, raw objective: 0 of 9 converge; the final objective is 0.8–11.4% above optimal
- Clarabel, lifted model: all 9 solved in at most 15 iterations, certified to 10^{-6}

What if we skip the lifting?

hand IPOPT the original objective, with exact automatic derivatives:

$$f(x) = \|Ax - b\|_2 + \gamma\|x\|_1$$

same 9 instances, same tolerance 10^{-9}, same 500-iteration limit:

- IPOPT, raw objective: 0 of 9 converge; the final objective is 0.8–11.4% above optimal
- Clarabel, lifted model: all 9 solved in at most 15 iterations, certified to 10^{-6}

Q: Why does IPOPT struggle here, when it did fine on the lifted model?

What if we skip the lifting?

hand IPOPT the original objective, with exact automatic derivatives:

$$f(x) = \|Ax - b\|_2 + \gamma\|x\|_1$$

same 9 instances, same tolerance 10^{-9}, same 500-iteration limit:

- IPOPT, raw objective: 0 of 9 converge; the final objective is 0.8–11.4% above optimal
- Clarabel, lifted model: all 9 solved in at most 15 iterations, certified to 10^{-6}

Q: Why does IPOPT struggle here, when it did fine on the lifted model?
A: $|x_i|$ has no derivative at 0, exactly where a sparse solution puts most coordinates; IPOPT's local quadratic model is wrong there.

What if we skip the lifting?

hand IPOPT the original objective, with exact automatic derivatives:

$$f(x) = \|Ax - b\|_2 + \gamma\|x\|_1$$

same 9 instances, same tolerance 10^{-9}, same 500-iteration limit:

- IPOPT, raw objective: 0 of 9 converge; the final objective is 0.8–11.4% above optimal
- Clarabel, lifted model: all 9 solved in at most 15 iterations, certified to 10^{-6}

Q: Why does IPOPT struggle here, when it did fine on the lifted model?
A: $|x_i|$ has no derivative at 0, exactly where a sparse solution puts most coordinates; IPOPT's local quadratic model is wrong there.

the lifting is the useful work; a conic solver accepts the lifted model natively

Convex function, different expressions

consider two expressions for the same function:

$$f(x) = \sqrt{1 + x^2} = \left\| \begin{pmatrix} 1 \\ x \end{pmatrix} \right\|_2 .$$

Q: How would you prove convexity from each expression?

Convex function, different expressions

consider two expressions for the same function:

$$f(x) = \sqrt{1 + x^2} = \left\| \begin{pmatrix} 1 \\ x \end{pmatrix} \right\|_2 .$$

Q: How would you prove convexity from each expression?

- on the left: a concave increasing function of a convex function; the usual composition rule is inconclusive
- on the right: a norm of an affine expression, hence convex

Convex function, different expressions

consider two expressions for the same function:

$$f(x) = \sqrt{1 + x^2} = \left\| \begin{pmatrix} 1 \\ x \end{pmatrix} \right\|_2 .$$

Q: How would you prove convexity from each expression?

- on the left: a concave increasing function of a convex function; the usual composition rule is inconclusive
- on the right: a norm of an affine expression, hence convex

Q: Could software accept the second expression and reject the first?

Convex function, different expressions

consider two expressions for the same function:

$$f(x) = \sqrt{1 + x^2} = \left\| \begin{pmatrix} 1 \\ x \end{pmatrix} \right\|_2 .$$

Q: How would you prove convexity from each expression?

- on the left: a concave increasing function of a convex function; the usual composition rule is inconclusive
- on the right: a norm of an affine expression, hence convex

Q: Could software accept the second expression and reject the first?
A: Yes. Convexity checks based on composition rules are sufficient, not necessary.

Disciplined convex programming

DCP builds expressions from known functions using valid composition rules

- convex nondecreasing function of a convex expression: convex
- convex nonincreasing function of a concave expression: convex
- convex function of an affine expression: convex

Disciplined convex programming

DCP builds expressions from known functions using valid composition rules

- convex nondecreasing function of a convex expression: convex
- convex nonincreasing function of a concave expression: convex
- convex function of an affine expression: convex

valid problem forms include

$$\text{minimize convex}, \qquad \text{maximize concave},$$

$$\text{convex} \leq \text{concave}, \qquad \text{affine} = \text{affine}.$$

Disciplined convex programming

DCP builds expressions from known functions using valid composition rules

- convex nondecreasing function of a convex expression: convex
- convex nonincreasing function of a concave expression: convex
- convex function of an affine expression: convex

valid problem forms include

$$\text{minimize convex}, \qquad \text{maximize concave},$$

$$\text{convex} \leq \text{concave}, \qquad \text{affine} = \text{affine}.$$

Q: If a DCP check fails, have we proved the problem is nonconvex?

Disciplined convex programming

DCP builds expressions from known functions using valid composition rules

- convex nondecreasing function of a convex expression: convex
- convex nonincreasing function of a concave expression: convex
- convex function of an affine expression: convex

valid problem forms include

$$\text{minimize convex}, \qquad \text{maximize concave},$$

$$\text{convex} \leq \text{concave}, \qquad \text{affine} = \text{affine}.$$

Q: If a DCP check fails, have we proved the problem is nonconvex?
A: No. We may need a different expression or a different formulation.

What does the modeling language do?

CVXPY checks convexity and translates the model into a solver's input form:

```
x = cp.Variable(n)
model = cp.Problem(cp.Minimize(
    cp.norm(A @ x - b, 2) + gamma * cp.norm1(x)))
data, _, _ = model.get_problem_data(cp.CLARABEL)
print(data["A"].shape)
print(data["dims"])
model.solve(solver=cp.CLARABEL)
```

What does the modeling language do?

CVXPY checks convexity and translates the model into a solver's input form:

```
x = cp.Variable(n)
model = cp.Problem(cp.Minimize(
    cp.norm(A @ x - b, 2) + gamma * cp.norm1(x)))
data, _, _ = model.get_problem_data(cp.CLARABEL)
print(data["A"].shape)
print(data["dims"])
model.solve(solver=cp.CLARABEL)
```

- **canonicalization**: introduce auxiliary variables and standard constraints
- a solver computes primal and dual variables for the resulting problem
- the modeling language maps the results back to the user's variables

What does the modeling language do?

CVXPY checks convexity and translates the model into a solver's input form:

```
x = cp.Variable(n)
model = cp.Problem(cp.Minimize(
    cp.norm(A @ x - b, 2) + gamma * cp.norm1(x)))
data, _, _ = model.get_problem_data(cp.CLARABEL)
print(data["A"].shape)
print(data["dims"])
model.solve(solver=cp.CLARABEL)
```

- **canonicalization**: introduce auxiliary variables and standard constraints
- a solver computes primal and dual variables for the resulting problem
- the modeling language maps the results back to the user's variables

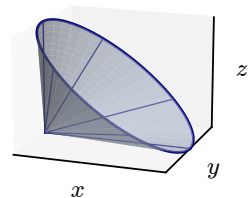
Q: Will its auxiliary variables match ours exactly? Do they need to?

Choosing a formulation

for $z \in \mathbb{R}^m$, both constraints describe $\|z\|_2 \leq t$:

$$(z, t) \in \mathcal{Q}^{m+1} \qquad \Longleftrightarrow \qquad \begin{bmatrix} tI_m & z \\ z^T & t \end{bmatrix} \succeq 0.$$

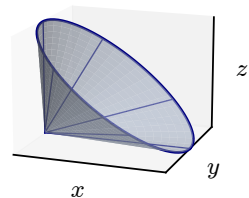
picture: 2×2 PSD matrices $\left[\begin{smallmatrix} x & y \\ y & z \end{smallmatrix}\right] \succeq 0$ form a second-order cone in rotated coordinates



Choosing a formulation

for $z \in \mathbb{R}^m$, both constraints describe $\|z\|_2 \leq t$:

$$(z, t) \in \mathcal{Q}^{m+1} \qquad \Longleftrightarrow \qquad \begin{bmatrix} tI_m & z \\ z^T & t \end{bmatrix} \succeq 0.$$



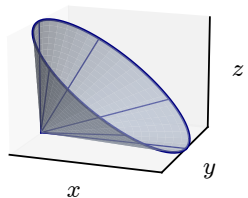
picture: 2×2 PSD matrices $\left[\begin{smallmatrix} x & y \\ y & z \end{smallmatrix}\right] \succeq 0$ form a second-order cone in rotated coordinates

- SOC form uses a cone of dimension $m + 1$
- PSD form uses a matrix of order $m + 1$: about $m^2/2$ entries
- a solver can exploit the SOC structure directly

Choosing a formulation

for $z \in \mathbb{R}^m$, both constraints describe $\|z\|_2 \leq t$:

$$(z, t) \in \mathcal{Q}^{m+1} \qquad \Longleftrightarrow \qquad \begin{bmatrix} tI_m & z \\ z^T & t \end{bmatrix} \succeq 0.$$



picture: 2×2 PSD matrices $\left[\begin{smallmatrix} x & y \\ y & z \end{smallmatrix}\right] \succeq 0$ form a second-order cone in rotated coordinates

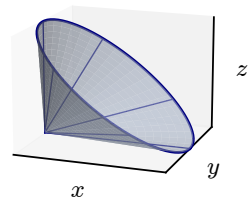
- SOC form uses a cone of dimension $m + 1$
- PSD form uses a matrix of order $m + 1$: about $m^2/2$ entries
- a solver can exploit the SOC structure directly

Q: Does a more general representation necessarily give a faster method?

Choosing a formulation

for $z \in \mathbb{R}^m$, both constraints describe $\|z\|_2 \leq t$:

$$(z, t) \in \mathcal{Q}^{m+1} \qquad \Longleftrightarrow \qquad \begin{bmatrix} tI_m & z \\ z^T & t \end{bmatrix} \succeq 0.$$



picture: 2×2 PSD matrices $\left[\begin{smallmatrix} x & y \\ y & z \end{smallmatrix}\right] \succeq 0$ form a second-order cone in rotated coordinates

- SOC form uses a cone of dimension $m + 1$
- PSD form uses a matrix of order $m + 1$: about $m^2/2$ entries
- a solver can exploit the SOC structure directly

Q: Does a more general representation necessarily give a faster method?

A: No. Preserve useful structure; compare size, sparsity, and solver support.

Clarabel QP benchmark: preserve quadratic structure

Clarabel can retain a convex quadratic objective or lift it to an SOCP:

$$\min_x \ \tfrac{1}{2}x^T Px + q^T x, \quad P \succeq 0, \qquad \text{or lift it to} \qquad \min_{x,t} \ t + q^T x, \quad \tfrac{1}{2}x^T Px \leq t.$$

Clarabel QP benchmark: preserve quadratic structure

Clarabel can retain a convex quadratic objective or lift it to an SOCP:

$$\min_x \ \tfrac{1}{2}x^T Px + q^T x, \quad P \succeq 0, \qquad \text{or lift it to} \qquad \min_{x,t} \ t + q^T x, \quad \tfrac{1}{2}x^T Px \le t.$$

138 QPs	normalized time	failure rate
Clarabel, native quadratic objective	1.00	10.9%
Clarabel, SOCP reformulation	11.82	43.5%
ECOS, SOCP reformulation	11.73	43.5%

ECOS: another conic solver; Maros–Mészáros: a standard QP test set; normalized shifted geometric mean time; failures charged 300 s

Goulart and Chen (2026), Fig. 2; solver-author benchmark, solver-specific default tolerances

Outline

Robust models

Why expose conic structure?

Conic form

From a model to a solver

Semidefinite programs

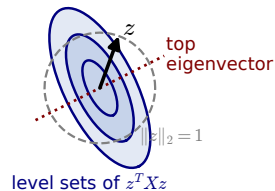
Conic duality

A relaxation: MaxCut

A bound in every direction

for a symmetric matrix X,

$$X \succeq 0 \quad \iff \quad z^T X z \geq 0 \quad \forall z.$$



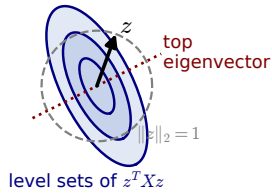
A bound in every direction

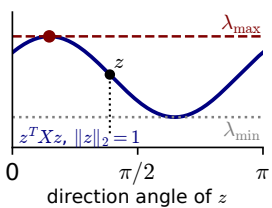
for a symmetric matrix X,

$$X \succeq 0 \iff z^T X z \geq 0 \quad \forall z.$$

hence

$$\begin{aligned} tI - X \succeq 0 &\iff z^T X z \leq t\|z\|_2^2 \quad \forall z \\ &\iff \lambda_{\max}(X) \leq t. \end{aligned}$$





A bound in every direction

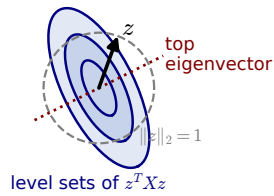
for a symmetric matrix X,

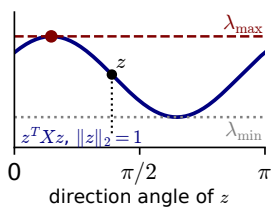
$$X \succeq 0 \quad \iff \quad z^T X z \geq 0 \quad \forall z.$$

hence

$$\begin{aligned} tI - X \succeq 0 \quad &\iff \quad z^T X z \leq t\|z\|_2^2 \quad \forall z \\ &\iff \quad \lambda_{\max}(X) \leq t. \end{aligned}$$

Q: Why is this infinite family of inequalities convex in (X, t)?





A bound in every direction

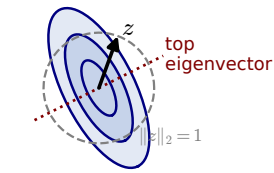
for a symmetric matrix X,

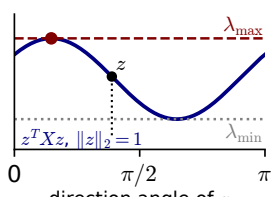
$$X \succeq 0 \quad \iff \quad z^T X z \geq 0 \quad \forall z.$$

hence

$$\begin{aligned} tI - X \succeq 0 \quad &\iff \quad z^T X z \leq t\|z\|_2^2 \quad \forall z \\ &\iff \quad \lambda_{\max}(X) \leq t. \end{aligned}$$

Q: Why is this infinite family of inequalities convex in (X, t)?
A: Each one is linear in (X, t); intersect them.





A bound in every direction

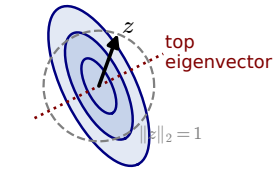
for a symmetric matrix X,

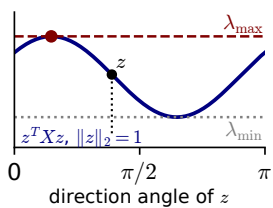
$$X \succeq 0 \quad \iff \quad z^T X z \geq 0 \quad \forall z.$$

hence

$$\begin{aligned} tI - X \succeq 0 \quad &\iff \quad z^T X z \leq t\|z\|_2^2 \quad \forall z \\ &\iff \quad \lambda_{\max}(X) \leq t. \end{aligned}$$

Q: Why is this infinite family of inequalities convex in (X, t)?
A: Each one is linear in (X, t); intersect them.
an SDP encodes bounds on a quadratic form in **every** direction





Designing for the worst direction

choose x to minimize the largest eigenvalue of a symmetric affine matrix:

$$X(x) = X_0 + \sum_{i=1}^{n} x_i X_i, \qquad X_i \in \mathbf{S}^m \text{ (symmetric } m \times m\text{)}.$$

Designing for the worst direction

choose x to minimize the largest eigenvalue of a symmetric affine matrix:

$$X(x) = X_0 + \sum_{i=1}^{n} x_i X_i, \qquad X_i \in \mathbf{S}^m \text{ (symmetric } m \times m\text{)}.$$

$$\min_{x \in C} \lambda_{\max}(X(x)) \quad \Longleftrightarrow \quad \begin{array}{ll} \text{minimize}_{x,t} & t \\ \text{subject to} & tI - X(x) \succeq 0, \quad x \in C. \end{array}$$

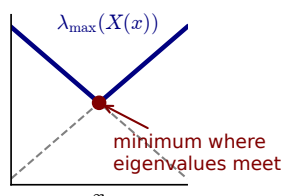
Designing for the worst direction

choose x to minimize the largest eigenvalue of a symmetric affine matrix:

$$X(x) = X_0 + \sum_{i=1}^{n} x_i X_i, \qquad X_i \in \mathbf{S}^m \text{ (symmetric } m \times m\text{)}.$$

$$\min_{x \in C} \lambda_{\max}(X(x)) \quad \Longleftrightarrow \quad \begin{array}{ll} \text{minimize}_{x,t} & t \\ \text{subject to} & tI - X(x) \succeq 0, \quad x \in C. \end{array}$$

this is an SDP when C is described by affine and PSD constraints



Designing for the worst direction

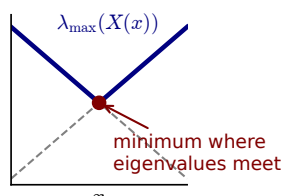
choose x to minimize the largest eigenvalue of a symmetric affine matrix:

$$X(x) = X_0 + \sum_{i=1}^{n} x_i X_i, \qquad X_i \in \mathbf{S}^m \text{ (symmetric } m \times m\text{)}.$$

$$\min_{x \in C} \lambda_{\max}(X(x)) \quad \Longleftrightarrow \quad \begin{array}{ll} \text{minimize}_{x,t} & t \\ \text{subject to} & tI - X(x) \succeq 0, \quad x \in C. \end{array}$$

this is an SDP when C is described by affine and PSD constraints

Q: Is this a relaxation? Must $X(x)$ itself be positive semidefinite?



Designing for the worst direction

choose x to minimize the largest eigenvalue of a symmetric affine matrix:

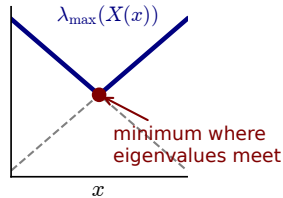
$$X(x) = X_0 + \sum_{i=1}^{n} x_i X_i, \qquad X_i \in \mathbf{S}^m \text{ (symmetric } m \times m\text{)}.$$

$$\min_{x \in C} \lambda_{\max}(X(x)) \quad \Longleftrightarrow \quad \begin{array}{ll} \text{minimize}_{x,t} & t \\ \text{subject to} & tI - X(x) \succeq 0, \quad x \in C. \end{array}$$

this is an SDP when C is described by affine and PSD constraints

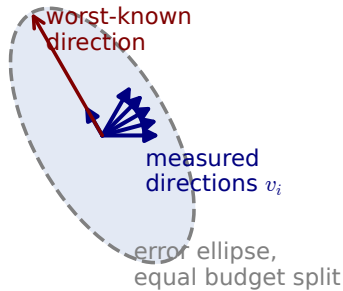
Q: Is this a relaxation? Must $X(x)$ itself be positive semidefinite?

A: It is exact, and $X(x)$ may be any symmetric matrix.



Experiment design: where should we measure?

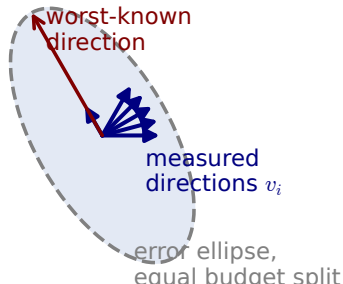
locate an unknown point $\theta \in \mathbb{R}^2$: sensor i reads $y = v_i^T \theta +$ noise (shorter v_i: noisier sensor)



Experiment design: where should we measure?

locate an unknown point $\theta \in \mathbb{R}^2$: sensor i reads $y = v_i^T\theta +$ noise (shorter v_i: noisier sensor)

- spend a fraction w_i of N readings on sensor i
- least-squares error $\approx \mathcal{N}\big(0, \frac{1}{N}M(w)^{-1}\big)$, $M(w) = \sum_i w_i v_i v_i^T$
- error ellipse $\{\delta : \delta^T M(w)\delta \le 1\}$: we learn θ along the directions we measure

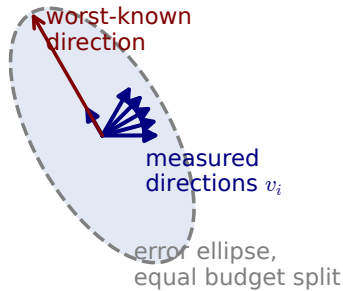


Experiment design: where should we measure?

locate an unknown point $\theta \in \mathbb{R}^2$: sensor i reads $y = v_i^T\theta +$ noise (shorter v_i: noisier sensor)

- spend a fraction w_i of N readings on sensor i
- least-squares error $\approx \mathcal{N}\big(0, \frac{1}{N}M(w)^{-1}\big)$, $M(w) = \sum_i w_i v_i v_i^T$
- error ellipse $\{\delta : \delta^T M(w)\delta \leq 1\}$: we learn θ along the directions we measure

Q: An equal split leaves one direction poorly known. Which?



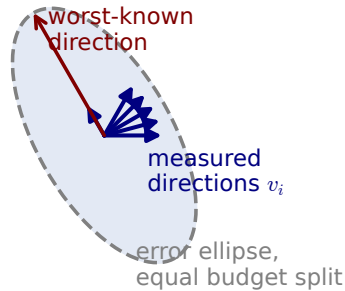
Experiment design: where should we measure?

locate an unknown point $\theta \in \mathbb{R}^2$: sensor i reads $y = v_i^T\theta + \text{noise}$ (shorter v_i: noisier sensor)

- spend a fraction w_i of N readings on sensor i
- least-squares error $\approx \mathcal{N}\big(0, \frac{1}{N}M(w)^{-1}\big)$, $M(w) = \sum_i w_i v_i v_i^T$
- error ellipse $\{\delta : \delta^T M(w)\delta \le 1\}$: we learn θ along the directions we measure

Q: An equal split leaves one direction poorly known. Which?

A: The long axis: the eigenvector for $\lambda_{\min}(M(w))$.



E-optimal design is an SDP

shrink the longest axis of the error ellipse:

$$\lambda_{\max}\big(M(w)^{-1}\big) = \frac{1}{\lambda_{\min}(M(w))} \quad \rightsquigarrow \quad \text{maximize } \lambda_{\min}(M(w))$$

E-optimal design is an SDP

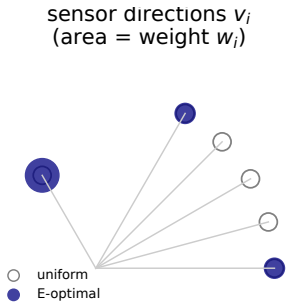
shrink the longest axis of the error ellipse:

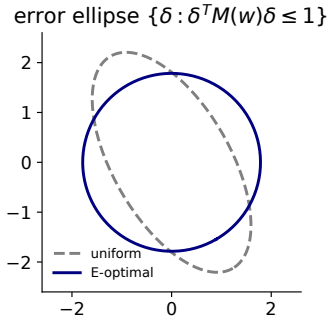
$$\lambda_{\max}\big(M(w)^{-1}\big) = \frac{1}{\lambda_{\min}(M(w))} \quad \rightsquigarrow \quad \text{maximize } \lambda_{\min}(M(w))$$

$M(w)$ is **affine** in w, and $\lambda_{\min}(M) \geq t \iff M - tI \succeq 0$:

$$\begin{array}{ll} \text{maximize}_{w,t} & t \\ \text{subject to} & M(w) - tI \succeq 0, \quad w \geq 0, \quad \mathbf{1}^T w = 1 \end{array}$$

Experiment design: what the SDP chooses





the six sensors from before: five similar, one noisier (shorter) in a new direction

- E-optimal design spends 58% of the budget on the noisy, neglected direction and drops three redundant experiments
- worst-direction standard deviation shrinks from 2.46 to 1.78

Outline

Robust models

Why expose conic structure?

Conic form

From a model to a solver

Semidefinite programs

Conic duality

A relaxation: MaxCut

Recall: the Lagrangian is built to be a lower bound

convex duality lecture: for $\min\{f_0(x) : f_i(x) \leq 0,\ x \in X\}$ and $\lambda \geq 0$,

$$\mathcal{L}(x, \lambda) = f_0(x) + \sum_i \lambda_i f_i(x) \ \leq \ f_0(x) \quad \text{for every feasible } x$$

Recall: the Lagrangian is built to be a lower bound

convex duality lecture: for $\min\{f_0(x) : f_i(x) \leq 0,\ x \in X\}$ and $\lambda \geq 0$,

$$\mathcal{L}(x, \lambda) = f_0(x) + \sum_i \lambda_i f_i(x) \ \leq \ f_0(x) \quad \text{for every feasible } x$$

- hence $g(\lambda) = \inf_{x \in X} \mathcal{L}(x, \lambda) \leq p^\star$
- the dual problem finds the best such bound
- the sign condition $\lambda \geq 0$ is **exactly** what makes each added term $\lambda_i f_i(x) \leq 0$ on feasible points

Recall: the Lagrangian is built to be a lower bound

convex duality lecture: for $\min\{f_0(x) : f_i(x) \leq 0,\ x \in X\}$ and $\lambda \geq 0$,

$$\mathcal{L}(x, \lambda) = f_0(x) + \sum_i \lambda_i f_i(x) \ \leq \ f_0(x) \quad \text{for every feasible } x$$

- hence $g(\lambda) = \inf_{x \in X} \mathcal{L}(x, \lambda) \leq p^\star$
- the dual problem finds the best such bound
- the sign condition $\lambda \geq 0$ is **exactly** what makes each added term $\lambda_i f_i(x) \leq 0$ on feasible points

now the constraint is $s = b - Ax \in K$; add the term $-\langle \lambda, s \rangle$

Recall: the Lagrangian is built to be a lower bound

convex duality lecture: for $\min\{f_0(x) : f_i(x) \leq 0,\ x \in X\}$ and $\lambda \geq 0$,

$$\mathcal{L}(x, \lambda) = f_0(x) + \sum_i \lambda_i f_i(x) \ \leq \ f_0(x) \quad \text{for every feasible } x$$

- hence $g(\lambda) = \inf_{x \in X} \mathcal{L}(x, \lambda) \leq p^\star$
- the dual problem finds the best such bound
- the sign condition $\lambda \geq 0$ is **exactly** what makes each added term $\lambda_i f_i(x) \leq 0$ on feasible points

now the constraint is $s = b - Ax \in K$; add the term $-\langle \lambda, s \rangle$

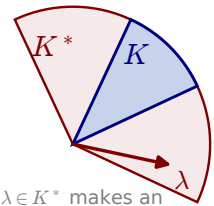
Q: Which multipliers λ make $-\langle \lambda, s \rangle \leq 0$ for every $s \in K$?

Dual cones

for a cone K, define its **dual cone**

$$K^* = \{\lambda : \langle \lambda, s \rangle \geq 0 \ \forall s \in K\}.$$

the multipliers that keep the Lagrangian a lower bound



every $\lambda \in K^*$ makes an angle $\leq 90^\circ$ with all of K

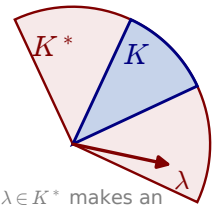
Dual cones

for a cone K, define its **dual cone**

$$K^* = \{\lambda : \langle \lambda, s \rangle \geq 0 \ \forall s \in K\}.$$

the multipliers that keep the Lagrangian a lower bound

Q: What is $(\mathbb{R}^m_+)^*$? What is $\{0\}^*$?



every $\lambda \in K^*$ makes an angle $\leq 90^\circ$ with all of K

Dual cones

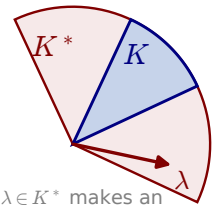
for a cone K, define its **dual cone**

$$K^* = \{\lambda : \langle \lambda, s \rangle \geq 0 \ \forall s \in K\}.$$

the multipliers that keep the Lagrangian a lower bound

Q: What is $(\mathbb{R}^m_+)^*$? What is $\{0\}^*$?

A: $\mathbb{R}^m_+$ (the usual $\lambda \geq 0$); everything (equalities: free).



every $\lambda \in K^*$ makes an angle $\leq 90^\circ$ with all of K

Dual cones

for a cone K, define its **dual cone**

$$K^* = \{\lambda : \langle \lambda, s \rangle \geq 0 \ \forall s \in K\}.$$

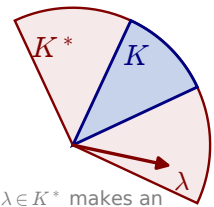
the multipliers that keep the Lagrangian a lower bound

Q: What is $(\mathbb{R}^m_+)^*$? What is $\{0\}^*$?

A: $\mathbb{R}^m_+$ (the usual $\lambda \geq 0$); everything (equalities: free).

self-dual ($K^* = K$), with $\langle X, Y \rangle = \mathbf{tr}(XY)$:

$$(\mathcal{Q}^{m+1})^* = \mathcal{Q}^{m+1}, \qquad (\mathbf{S}^m_+)^* = \mathbf{S}^m_+$$

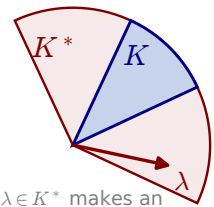


every $\lambda \in K^*$ makes an angle $\leq 90^\circ$ with all of K

Dual cones

for a cone K, define its **dual cone**

$$K^* = \{\lambda : \langle \lambda, s \rangle \geq 0 \ \forall s \in K\}.$$



every $\lambda \in K^*$ makes an angle $\leq 90^\circ$ with all of K

the multipliers that keep the Lagrangian a lower bound

Q: What is $(\mathbb{R}^m_+)^*$? What is $\{0\}^*$?

A: $\mathbb{R}^m_+$ (the usual $\lambda \geq 0$); everything (equalities: free).

self-dual ($K^* = K$), with $\langle X, Y \rangle = \mathbf{tr}(XY)$:

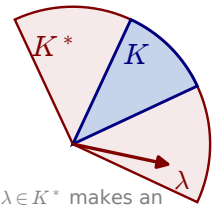
$$(\mathcal{Q}^{m+1})^* = \mathcal{Q}^{m+1}, \qquad (\mathbf{S}^m_+)^* = \mathbf{S}^m_+$$

Q: Why is $\mathcal{Q}$ self-dual? Use Cauchy–Schwarz, as in the robust model.

Dual cones

for a cone K, define its **dual cone**

$$K^* = \{\lambda : \langle \lambda, s \rangle \geq 0 \ \forall s \in K\}.$$



every $\lambda \in K^*$ makes an angle $\leq 90^\circ$ with all of K

the multipliers that keep the Lagrangian a lower bound

Q: What is $(\mathbb{R}^m_+)^*$? What is $\{0\}^*$?

A: $\mathbb{R}^m_+$ (the usual $\lambda \geq 0$); everything (equalities: free).

self-dual ($K^* = K$), with $\langle X, Y \rangle = \mathbf{tr}(XY)$:

$$(\mathcal{Q}^{m+1})^* = \mathcal{Q}^{m+1}, \qquad (\mathbf{S}^m_+)^* = \mathbf{S}^m_+$$

Q: Why is $\mathcal{Q}$ self-dual? Use Cauchy–Schwarz, as in the robust model.

$(z, t), (u, \tau) \in \mathcal{Q}$: $z^T u + t\tau \geq -\|z\|_2 \|u\|_2 + t\tau \geq 0$. Conversely, test (u, τ) against $(-u, \|u\|_2) \in \mathcal{Q}$ to get $\tau \geq \|u\|_2$.

Conic duality: the same recipe

minimize_x	$c^T x$	maximize_λ	$-b^T \lambda$
subject to	$b - Ax \in K$	subject to	$c + A^T \lambda = 0, \quad \lambda \in K^*.$

Conic duality: the same recipe

minimize_x	$c^T x$	maximize_λ	$-b^T \lambda$
subject to	$b - Ax \in K$	subject to	$c + A^T \lambda = 0, \quad \lambda \in K^*.$

1. attach $\lambda \in K^*$ to $b - Ax \in K$ (block by block for a product cone)
2. lower bound: $\mathcal{L}(x, \lambda) = c^T x - \langle \lambda, b - Ax \rangle \leq c^T x$ for feasible x
3. minimize over x: $\mathcal{L}(x, \lambda) = -b^T \lambda + (c + A^T \lambda)^T x$

Conic duality: the same recipe

minimize_x	$c^T x$	maximize_λ	$-b^T \lambda$
subject to	$b - Ax \in K$	subject to	$c + A^T \lambda = 0, \quad \lambda \in K^*.$

1. attach $\lambda \in K^*$ to $b - Ax \in K$ (block by block for a product cone)
2. lower bound: $\mathcal{L}(x, \lambda) = c^T x - \langle \lambda, b - Ax \rangle \leq c^T x$ for feasible x
3. minimize over x: $\mathcal{L}(x, \lambda) = -b^T \lambda + (c + A^T \lambda)^T x$

Q: Why must $c + A^T \lambda = 0$ for $\inf_x \mathcal{L}(x, \lambda)$ to be finite?

Conic duality: the same recipe

minimize_x	$c^T x$	maximize_λ	$-b^T \lambda$
subject to	$b - Ax \in K$	subject to	$c + A^T \lambda = 0, \quad \lambda \in K^*.$

1. attach $\lambda \in K^*$ to $b - Ax \in K$ (block by block for a product cone)
2. lower bound: $\mathcal{L}(x, \lambda) = c^T x - \langle \lambda, b - Ax \rangle \leq c^T x$ for feasible x
3. minimize over x: $\mathcal{L}(x, \lambda) = -b^T \lambda + (c + A^T \lambda)^T x$

Q: Why must $c + A^T \lambda = 0$ for $\inf_x \mathcal{L}(x, \lambda)$ to be finite?
A: Otherwise an unrestricted x can drive the linear term to $-\infty$.

Conic duality: the same recipe

minimize_x	$c^T x$	maximize_λ	$-b^T \lambda$
subject to	$b - Ax \in K$	subject to	$c + A^T \lambda = 0, \quad \lambda \in K^*.$

1. attach $\lambda \in K^*$ to $b - Ax \in K$ (block by block for a product cone)
2. lower bound: $\mathcal{L}(x, \lambda) = c^T x - \langle \lambda, b - Ax \rangle \leq c^T x$ for feasible x
3. minimize over x: $\mathcal{L}(x, \lambda) = -b^T \lambda + (c + A^T \lambda)^T x$

Q: Why must $c + A^T \lambda = 0$ for $\inf_x \mathcal{L}(x, \lambda)$ to be finite?
A: Otherwise an unrestricted x can drive the linear term to $-\infty$.
with matrix constraints, replace A^T by the adjoint of the linear map; for a maximization, build an **upper** bound instead

Exercise: the dual of square-root Lasso

$$\begin{array}{ll} \text{minimize}_{x,r} & \|r\|_2 + \gamma\|x\|_1 \\ \text{subject to} & r = Ax - b \end{array}$$

Q: Derive the dual.

- attach a multiplier y to $r = Ax - b$. What sign constraint does y need?
- minimize the Lagrangian over r and over x separately
- which values of y give a finite lower bound?

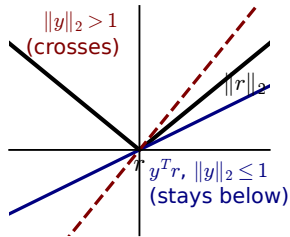
Square-root Lasso: the dual

$$\mathcal{L}(x, r, y) = \left(\|r\|_2 - y^T r\right) + \left(\gamma\|x\|_1 + y^T A x\right) - b^T y$$

Square-root Lasso: the dual

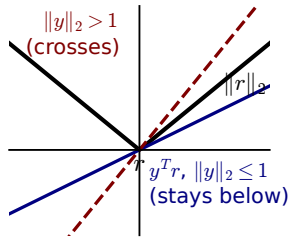
$$\mathcal{L}(x, r, y) = \left(\|r\|_2 - y^T r\right) + \left(\gamma\|x\|_1 + y^T A x\right) - b^T y$$

- $\inf_r \left(\|r\|_2 - y^T r\right) = 0$ if $\|y\|_2 \leq 1$, and $-\infty$ otherwise
- $\inf_x \left(\gamma\|x\|_1 + (A^T y)^T x\right) = 0$ if $\|A^T y\|_\infty \leq \gamma$, and $-\infty$ otherwise



Square-root Lasso: the dual

$$\mathcal{L}(x, r, y) = \left(\|r\|_2 - y^T r\right) + \left(\gamma\|x\|_1 + y^T A x\right) - b^T y$$



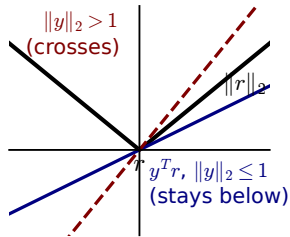
- $\inf_r \left(\|r\|_2 - y^T r\right) = 0$ if $\|y\|_2 \le 1$, and $-\infty$ otherwise
- $\inf_x \left(\gamma\|x\|_1 + (A^T y)^T x\right) = 0$ if $\|A^T y\|_\infty \le \gamma$, and $-\infty$ otherwise

$$\begin{array}{ll} \text{maximize}_y & -b^T y \\ \text{subject to} & \|y\|_2 \le 1, \quad \|A^T y\|_\infty \le \gamma \end{array}$$

dual norms $\|y\|_* = \sup_{\|r\| \le 1} y^T r$ appear: $\ell_2 \leftrightarrow \ell_2$, $\ell_1 \leftrightarrow \ell_\infty$

Square-root Lasso: the dual

$$\mathcal{L}(x, r, y) = \left(\|r\|_2 - y^T r\right) + \left(\gamma\|x\|_1 + y^T A x\right) - b^T y$$



- $\inf_r \left(\|r\|_2 - y^T r\right) = 0$ if $\|y\|_2 \le 1$, and $-\infty$ otherwise
- $\inf_x \left(\gamma\|x\|_1 + (A^T y)^T x\right) = 0$ if $\|A^T y\|_\infty \le \gamma$, and $-\infty$ otherwise

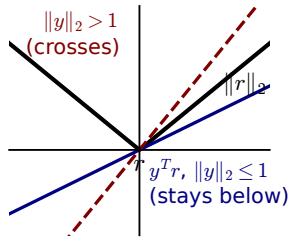
$$\begin{array}{ll} \text{maximize}_y & -b^T y \\ \text{subject to} & \|y\|_2 \le 1, \quad \|A^T y\|_\infty \le \gamma \end{array}$$

dual norms $\|y\|_* = \sup_{\|r\| \le 1} y^T r$ appear: $\ell_2 \leftrightarrow \ell_2$, $\ell_1 \leftrightarrow \ell_\infty$

Q: How do dual norms relate to dual cones?

Square-root Lasso: the dual

$$\mathcal{L}(x, r, y) = \left(\|r\|_2 - y^T r\right) + \left(\gamma\|x\|_1 + y^T A x\right) - b^T y$$



- $\inf_r \left(\|r\|_2 - y^T r\right) = 0$ if $\|y\|_2 \le 1$, and $-\infty$ otherwise
- $\inf_x \left(\gamma\|x\|_1 + (A^T y)^T x\right) = 0$ if $\|A^T y\|_\infty \le \gamma$, and $-\infty$ otherwise

$$\begin{array}{ll} \text{maximize}_y & -b^T y \\ \text{subject to} & \|y\|_2 \le 1, \quad \|A^T y\|_\infty \le \gamma \end{array}$$

dual norms $\|y\|_* = \sup_{\|r\|\le 1} y^T r$ appear: $\ell_2 \leftrightarrow \ell_2$, $\ell_1 \leftrightarrow \ell_\infty$

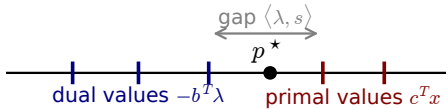
Q: How do dual norms relate to dual cones?

A: The dual of the norm cone $\{(r, t) : \|r\| \le t\}$ is $\{(y, \tau) : \|y\|_* \le \tau\}$.

The gap is a certificate

for **primal and dual feasible** x, λ, let $s = b - Ax \in K$:

$$0 \leq c^T x - p^\star \leq c^T x + b^T \lambda = \langle \lambda, s \rangle.$$

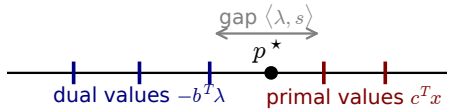


The gap is a certificate

for **primal and dual feasible** x, λ, let $s = b - Ax \in K$:

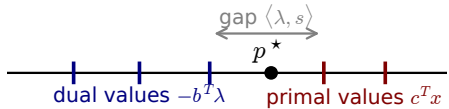
$$0 \leq c^T x - p^\star \leq c^T x + b^T \lambda = \langle \lambda, s \rangle.$$

Q: If $\langle \lambda, s \rangle = 0$, what have we proved?



The gap is a certificate

for **primal and dual feasible** x, λ, let $s = b - Ax \in K$:



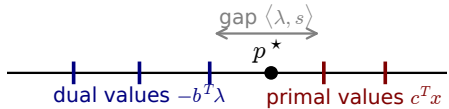
$$0 \leq c^T x - p^\star \leq c^T x + b^T \lambda = \langle \lambda, s \rangle.$$

Q: If $\langle \lambda, s \rangle = 0$, what have we proved?

A: x and λ are optimal: complementary slackness certifies optimality.

The gap is a certificate

for **primal and dual feasible** x, λ, let $s = b - Ax \in K$:



$$0 \leq c^T x - p^\star \leq c^T x + b^T \lambda = \langle \lambda, s \rangle.$$

Q: If $\langle \lambda, s \rangle = 0$, what have we proved?

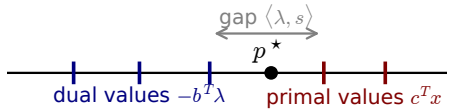
A: x and λ are optimal: complementary slackness certifies optimality.

a sufficient condition for strong duality (Slater): the primal value is finite, and some x satisfies $b - Ax \in$ **relint** K

- with explicit equalities, satisfy them exactly; strict feasibility for the other blocks

The gap is a certificate

for **primal and dual feasible** x, λ, let $s = b - Ax \in K$:



$$0 \leq c^T x - p^\star \leq c^T x + b^T \lambda = \langle \lambda, s \rangle.$$

Q: If $\langle \lambda, s \rangle = 0$, what have we proved?

A: x and λ are optimal: complementary slackness certifies optimality.

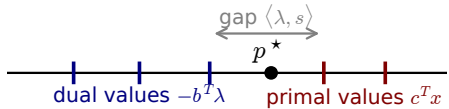
a sufficient condition for strong duality (Slater): the primal value is finite, and some x satisfies $b - Ax \in$ **relint** K

- with explicit equalities, satisfy them exactly; strict feasibility for the other blocks

to check optimality of a computed solution: check primal residuals, dual residuals ($c + A^T\lambda \approx 0$, $\lambda \in K^*$), and the optimality gap

The gap is a certificate

for **primal and dual feasible** x, λ, let $s = b - Ax \in K$:



$$0 \le c^T x - p^\star \le c^T x + b^T \lambda = \langle \lambda, s \rangle.$$

Q: If $\langle \lambda, s \rangle = 0$, what have we proved?

A: x and λ are optimal: complementary slackness certifies optimality.

a sufficient condition for strong duality (Slater): the primal value is finite, and some x satisfies $b - Ax \in$ **relint** K

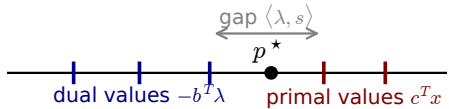
- with explicit equalities, satisfy them exactly; strict feasibility for the other blocks

to check optimality of a computed solution: check primal residuals, dual residuals ($c + A^T\lambda \approx 0$, $\lambda \in K^*$), and the optimality gap

Q: Can a tiny reported gap certify an infeasible decision?

The gap is a certificate

for **primal and dual feasible** x, λ, let $s = b - Ax \in K$:



$$0 \le c^T x - p^\star \le c^T x + b^T \lambda = \langle \lambda, s \rangle.$$

Q: If $\langle \lambda, s \rangle = 0$, what have we proved?

A: x and λ are optimal: complementary slackness certifies optimality.

a sufficient condition for strong duality (Slater): the primal value is finite, and some x satisfies $b - Ax \in$ **relint** K

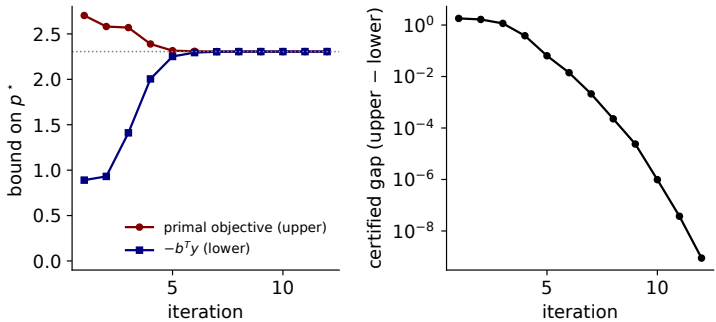
- with explicit equalities, satisfy them exactly; strict feasibility for the other blocks

to check optimality of a computed solution: check primal residuals, dual residuals ($c + A^T\lambda \approx 0$, $\lambda \in K^*$), and the optimality gap

Q: Can a tiny reported gap certify an infeasible decision?

A: No. The gap bound assumed both primal and dual feasibility.

Watching the certificate form



square-root Lasso, $(m, n) = (300, 150)$; at **every** Clarabel iterate:

- upper bound: the original objective at x_k (every x is feasible)
- lower bound: $-b^T y_k$, after scaling y_k so that $\|y\|_2 \leq 1$ and $\|A^T y\|_\infty \leq \gamma$

the certified gap falls from 1.8 to 10^{-9} in 12 iterations

Certificates of infeasibility

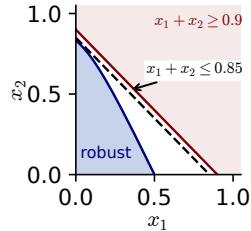
robust production ($\rho = 2$) with a minimum output $x_1 + x_2 \geq 0.9$

Certificates of infeasibility

robust production ($\rho = 2$) with a minimum output $x_1 + x_2 \geq 0.9$

conic Farkas: if $\lambda \in K^*$, $A^T\lambda = 0$, $b^T\lambda < 0$, then no x has $b - Ax \in K$, since a feasible x would give $0 \leq \langle \lambda, b - Ax \rangle = b^T\lambda < 0$ (the Lagrangian bound again)

Certificates of infeasibility



robust production ($\rho = 2$) with a minimum output $x_1 + x_2 \geq 0.9$

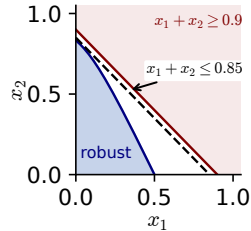
conic Farkas: if $\lambda \in K^*$, $A^T\lambda = 0$, $b^T\lambda < 0$, then no x has $b - Ax \in K$, since a feasible x would give $0 \leq \langle \lambda, b - Ax \rangle = b^T\lambda < 0$ (the Lagrangian bound again)

a certificate you can check (here $\rho P^T x = (x_1, 0.2x_2)$):

$$0.15(x_1 + x_2) = \begin{pmatrix} 0.15 \\ 0.75 \end{pmatrix}^T \begin{pmatrix} x_1 \\ 0.2x_2 \end{pmatrix} \leq 0.765 \left\| \begin{pmatrix} x_1 \\ 0.2x_2 \end{pmatrix} \right\|_2 \leq 0.85(1 - x_1 - x_2)$$

so $x_1 + x_2 \leq 0.85 < 0.9$

Certificates of infeasibility



robust production ($\rho = 2$) with a minimum output $x_1 + x_2 \geq 0.9$

conic Farkas: if $\lambda \in K^*$, $A^T\lambda = 0$, $b^T\lambda < 0$, then no x has $b - Ax \in K$, since a feasible x would give $0 \leq \langle \lambda, b - Ax \rangle = b^T\lambda < 0$ (the Lagrangian bound again)

a certificate you can check (here $\rho P^T x = (x_1, 0.2x_2)$):

$$0.15(x_1 + x_2) = \begin{pmatrix} 0.15 \\ 0.75 \end{pmatrix}^T \begin{pmatrix} x_1 \\ 0.2x_2 \end{pmatrix} \leq 0.765 \left\| \begin{pmatrix} x_1 \\ 0.2x_2 \end{pmatrix} \right\|_2 \leq 0.85(1 - x_1 - x_2)$$

so $x_1 + x_2 \leq 0.85 < 0.9$

- Clarabel: `PrimalInfeasible` after 8 iterations, with a certificate like this one
- IPOPT (squared form): "infeasible problem detected" after 16 iterations; not a proof

Outline

Robust models

Why expose conic structure?

Conic form

From a model to a solver

Semidefinite programs

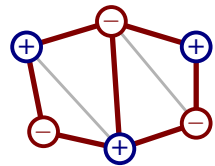
Conic duality

A relaxation: MaxCut

Lifting products: MaxCut

a **cut** splits the vertices of a graph into two groups: assign each vertex a sign $x_i \in \{-1, 1\}$

$$\frac{1 - x_i x_j}{2} = \begin{cases} 1 & \text{if edge } \{i, j\} \text{ crosses the cut} \\ 0 & \text{otherwise.} \end{cases}$$



7 of 9 edges cross the cut

Lifting products: MaxCut

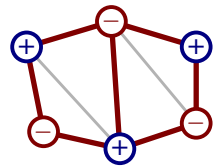
a **cut** splits the vertices of a graph into two groups: assign each vertex a sign $x_i \in \{-1, 1\}$

$$\frac{1 - x_i x_j}{2} = \begin{cases} 1 & \text{if edge } \{i, j\} \text{ crosses the cut} \\ 0 & \text{otherwise.} \end{cases}$$

for edge weights $w_{ij} \geq 0$, MaxCut is

$$\max_{x_i \in \{-1,1\}} \sum_{\{i,j\} \in E} w_{ij} \frac{1 - x_i x_j}{2}.$$

each undirected edge is counted once



7 of 9 edges cross the cut

Lifting products: MaxCut

a **cut** splits the vertices of a graph into two groups: assign each vertex a sign $x_i \in \{-1, 1\}$

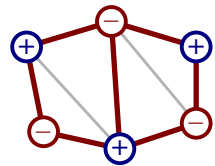
$$\frac{1 - x_i x_j}{2} = \begin{cases} 1 & \text{if edge } \{i, j\} \text{ crosses the cut} \\ 0 & \text{otherwise.} \end{cases}$$

for edge weights $w_{ij} \geq 0$, MaxCut is

$$\max_{x_i \in \{-1,1\}} \sum_{\{i,j\} \in E} w_{ij} \frac{1 - x_i x_j}{2}.$$

each undirected edge is counted once

Q: Can we make the objective linear by introducing a matrix?



7 of 9 edges cross the cut

Lifting products: MaxCut

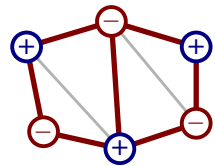
a **cut** splits the vertices of a graph into two groups:
assign each vertex a sign $x_i \in \{-1, 1\}$

$$\frac{1 - x_i x_j}{2} = \begin{cases} 1 & \text{if edge } \{i,j\} \text{ crosses the cut} \\ 0 & \text{otherwise.} \end{cases}$$

for edge weights $w_{ij} \geq 0$, MaxCut is

$$\max_{x_i \in \{-1,1\}} \sum_{\{i,j\} \in E} w_{ij} \frac{1 - x_i x_j}{2}.$$

each undirected edge is counted once



7 of 9 edges cross the cut

Q: Can we make the objective linear by introducing a matrix?
let $X = xx^T$. Then $X_{ij} = x_i x_j$, $X \succeq 0$, $X_{ii} = 1$, and $\mathrm{Rank}(X) = 1$

Where does the relaxation happen?

the **exact** lifted problem is

$$\begin{array}{ll} \text{maximize}_X & \displaystyle\sum_{\{i,j\}\in E} w_{ij}\frac{1-X_{ij}}{2} \\ \text{subject to} & X \succeq 0, \quad X_{ii} = 1, \quad \mathrm{Rank}(X) = 1. \end{array}$$

Where does the relaxation happen?

the **exact** lifted problem is

$$\begin{array}{ll} \text{maximize}_X & \displaystyle\sum_{\{i,j\}\in E} w_{ij}\frac{1-X_{ij}}{2} \\ \text{subject to} & X \succeq 0, \quad X_{ii} = 1, \quad \mathrm{Rank}(X) = 1. \end{array}$$

Q: Which constraint is nonconvex? What if we remove it?

Where does the relaxation happen?

the **exact** lifted problem is

$$\begin{array}{ll} \text{maximize}_X & \displaystyle\sum_{\{i,j\}\in E} w_{ij}\frac{1-X_{ij}}{2} \\ \text{subject to} & X \succeq 0, \quad X_{ii} = 1, \quad \mathrm{Rank}(X) = 1. \end{array}$$

Q: Which constraint is nonconvex? What if we remove it?
dropping rank gives an SDP **relaxation**: every cut is still feasible

Where does the relaxation happen?

the **exact** lifted problem is

$$\begin{array}{ll} \text{maximize}_X & \displaystyle\sum_{\{i,j\}\in E} w_{ij}\frac{1-X_{ij}}{2} \\ \text{subject to} & X \succeq 0, \quad X_{ii} = 1, \quad \mathrm{Rank}(X) = 1. \end{array}$$

Q: Which constraint is nonconvex? What if we remove it?
dropping rank gives an SDP **relaxation**: every cut is still feasible

Q: Does its optimal value give an upper or lower bound on MaxCut?

Where does the relaxation happen?

the **exact** lifted problem is

$$\begin{array}{ll} \text{maximize}_X & \displaystyle\sum_{\{i,j\}\in E} w_{ij}\frac{1-X_{ij}}{2} \\ \text{subject to} & X \succeq 0, \quad X_{ii} = 1, \quad \mathrm{Rank}(X) = 1. \end{array}$$

Q: Which constraint is nonconvex? What if we remove it?
dropping rank gives an SDP **relaxation**: every cut is still feasible

Q: Does its optimal value give an upper or lower bound on MaxCut?
A: An upper bound: we maximize over a larger set.

Where does the relaxation happen?

the **exact** lifted problem is

$$\begin{array}{ll} \text{maximize}_X & \displaystyle\sum_{\{i,j\}\in E} w_{ij}\frac{1-X_{ij}}{2} \\ \text{subject to} & X \succeq 0, \quad X_{ii} = 1, \quad \mathrm{Rank}(X) = 1. \end{array}$$

Q: Which constraint is nonconvex? What if we remove it?
dropping rank gives an SDP **relaxation**: every cut is still feasible

Q: Does its optimal value give an upper or lower bound on MaxCut?
A: An upper bound: we maximize over a larger set.
unlike an epigraph reformulation, this step may change the optimal value

the same lift works for any quadratic objective over $x \in \{-1, 1\}^n$: the **Shor relaxation**

A triangle exposes the gap

three vertices, all three edges present, every edge has weight 1

A triangle exposes the gap

three vertices, all three edges present, every edge has weight 1

Q: How many edges can a cut cross?

A triangle exposes the gap

three vertices, all three edges present, every edge has weight 1

Q: How many edges can a cut cross?

A: At most two. A triangle cannot have opposite signs on every edge.

A triangle exposes the gap

three vertices, all three edges present, every edge has weight 1

Q: How many edges can a cut cross?

A: At most two. A triangle cannot have opposite signs on every edge.

consider

$$X = \begin{bmatrix} 1 & -1/2 & -1/2 \\ -1/2 & 1 & -1/2 \\ -1/2 & -1/2 & 1 \end{bmatrix} = \frac{3}{2} I - \frac{1}{2} \mathbf{1}\mathbf{1}^T.$$

- eigenvalues $0, 3/2, 3/2$: PSD, with diagonal 1
- SDP objective $3 \cdot (1 + 1/2)/2 = 9/4 > 2$

A triangle exposes the gap

three vertices, all three edges present, every edge has weight 1

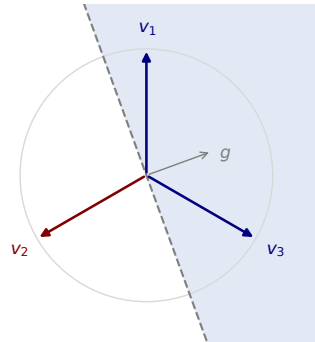
Q: How many edges can a cut cross?

A: At most two. A triangle cannot have opposite signs on every edge.

consider

$$X = \begin{bmatrix} 1 & -1/2 & -1/2 \\ -1/2 & 1 & -1/2 \\ -1/2 & -1/2 & 1 \end{bmatrix} = \frac{3}{2} I - \frac{1}{2} \mathbf{1}\mathbf{1}^T.$$

- eigenvalues $0, 3/2, 3/2$: PSD, with diagonal 1
- SDP objective $3 \cdot (1 + 1/2)/2 = 9/4 > 2$



$X = VV^T$: unit vectors 120° apart

A triangle exposes the gap

three vertices, all three edges present, every edge has weight 1

Q: How many edges can a cut cross?

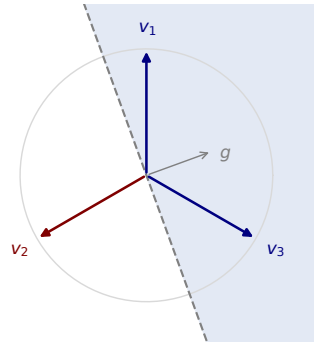
A: At most two. A triangle cannot have opposite signs on every edge.

consider

$$X = \begin{bmatrix} 1 & -1/2 & -1/2 \\ -1/2 & 1 & -1/2 \\ -1/2 & -1/2 & 1 \end{bmatrix} = \frac{3}{2} I - \frac{1}{2} \mathbf{1}\mathbf{1}^T.$$

- eigenvalues $0, 3/2, 3/2$: PSD, with diagonal 1
- SDP objective $3 \cdot (1 + 1/2)/2 = 9/4 > 2$

Q: What prevents this X from representing a cut?



$X = VV^T$: unit vectors 120° apart

A triangle exposes the gap

three vertices, all three edges present, every edge has weight 1

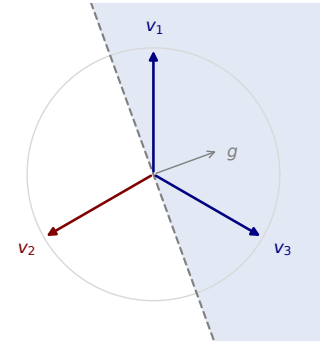
Q: How many edges can a cut cross?

A: At most two. A triangle cannot have opposite signs on every edge.

consider

$$X = \begin{bmatrix} 1 & -1/2 & -1/2 \\ -1/2 & 1 & -1/2 \\ -1/2 & -1/2 & 1 \end{bmatrix} = \frac{3}{2} I - \frac{1}{2} \mathbf{1}\mathbf{1}^T.$$

- eigenvalues $0, 3/2, 3/2$: PSD, with diagonal 1
- SDP objective $3 \cdot (1 + 1/2)/2 = 9/4 > 2$



$X = VV^T$: unit vectors 120° apart

Q: What prevents this X from representing a cut?

A: It has rank two: the vectors live in a plane. It cannot equal xx^T.

Recovering a decision

factor a feasible SDP matrix as $X = VV^T$; its rows v_i are unit vectors

- if X has rank one, recover signs x_i exactly

Recovering a decision

factor a feasible SDP matrix as $X = VV^T$; its rows v_i are unit vectors

- if X has rank one, recover signs x_i exactly
- otherwise, choose a random direction g and use $x_i = \mathbf{sign}(v_i^T g)$
- assign either sign in a tie; evaluate the resulting cut

Recovering a decision

factor a feasible SDP matrix as $X = VV^T$; its rows v_i are unit vectors

- if X has rank one, recover signs x_i exactly
- otherwise, choose a random direction g and use $x_i = \mathbf{sign}(v_i^T g)$
- assign either sign in a tie; evaluate the resulting cut

$$\text{value of any cut} \;\leq\; \text{best cut value} \;\leq\; \text{SDP optimal value.}$$

Recovering a decision

factor a feasible SDP matrix as $X = VV^T$; its rows v_i are unit vectors

- if X has rank one, recover signs x_i exactly
- otherwise, choose a random direction g and use $x_i = \mathbf{sign}(v_i^T g)$
- assign either sign in a tie; evaluate the resulting cut

$$\text{value of any cut} \;\leq\; \text{best cut value} \;\leq\; \text{SDP optimal value.}$$

Goemans–Williamson (1995): for $w \geq 0$ and Gaussian g,

$$\mathbf{E}[\text{rounded cut}] \;\geq\; 0.878 \cdot \text{SDP value} \;\geq\; 0.878 \cdot \text{best cut}$$

Recovering a decision

factor a feasible SDP matrix as $X = VV^T$; its rows v_i are unit vectors

- if X has rank one, recover signs x_i exactly
- otherwise, choose a random direction g and use $x_i = \mathbf{sign}(v_i^T g)$
- assign either sign in a tie; evaluate the resulting cut

$$\text{value of any cut} \;\leq\; \text{best cut value} \;\leq\; \text{SDP optimal value.}$$

Goemans–Williamson (1995): for $w \geq 0$ and Gaussian g,

$$\mathbf{E}[\text{rounded cut}] \;\geq\; 0.878 \cdot \text{SDP value} \;\geq\; 0.878 \cdot \text{best cut}$$

Q: Does a feasible matrix alone give an upper bound on the best cut?

Recovering a decision

factor a feasible SDP matrix as $X = VV^T$; its rows v_i are unit vectors

- if X has rank one, recover signs x_i exactly
- otherwise, choose a random direction g and use $x_i = \mathbf{sign}(v_i^T g)$
- assign either sign in a tie; evaluate the resulting cut

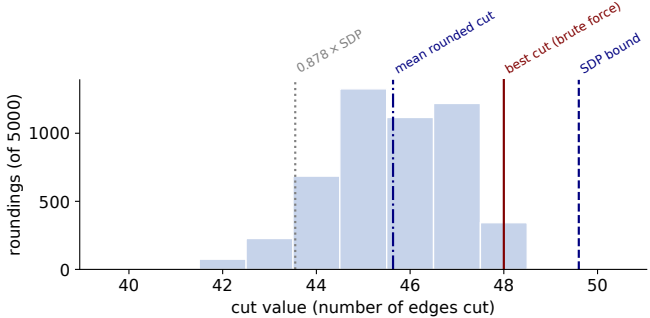
$$\text{value of any cut} \;\leq\; \text{best cut value} \;\leq\; \text{SDP optimal value.}$$

Goemans–Williamson (1995): for $w \geq 0$ and Gaussian g,

$$\mathbf{E}[\text{rounded cut}] \;\geq\; 0.878 \cdot \text{SDP value} \;\geq\; 0.878 \cdot \text{best cut}$$

Q: Does a feasible matrix alone give an upper bound on the best cut?
A: No. We need the SDP optimum, or a certified upper bound from its dual.

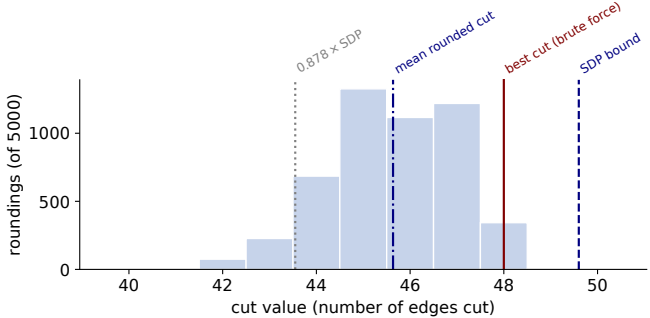
How good is rounding?



random graph: 20 vertices, 67 edges; brute force checks all 2^{19} cuts

- best cut 48; SDP bound 49.6: the relaxation is within 4%
- 5000 random roundings: mean 45.6 (0.92× bound); 7% find a best cut

How good is rounding?



random graph: 20 vertices, 67 edges; brute force checks all 2^{19} cuts

- best cut 48; SDP bound 49.6: the relaxation is within 4%
- 5000 random roundings: mean 45.6 (0.92× bound); 7% find a best cut

a relaxation is useful when it provides a bound, a good decision, or both

Example: the dual of the MaxCut SDP

write the objective as $\langle C, X \rangle$ (using $X_{ii} = 1$), where $C = \frac{1}{4}L$ and $L = \mathbf{Diag}(W\mathbf{1}) - W$ is the graph Laplacian (W: edge weights; $\mathbf{Diag}(y)$: diagonal matrix with diagonal y)

$$\begin{array}{ll} \text{maximize}_X & \langle C, X \rangle \\ \text{subject to} & X_{ii} = 1, \quad X \succeq 0 \end{array}$$

Example: the dual of the MaxCut SDP

write the objective as $\langle C, X \rangle$ (using $X_{ii} = 1$), where $C = \frac{1}{4}L$ and $L = \mathbf{Diag}(W\mathbf{1}) - W$ is the graph Laplacian (W: edge weights; $\mathbf{Diag}(y)$: diagonal matrix with diagonal y)

$$\begin{array}{ll} \text{maximize}_X & \langle C, X \rangle \\ \text{subject to} & X_{ii} = 1, \quad X \succeq 0 \end{array}$$

we maximize, so build an **upper** bound: for $y \in \mathbb{R}^n$ and $Z \succeq 0$,

$$\mathcal{L}(X, y, Z) = \langle C, X \rangle + \sum_i y_i(1 - X_{ii}) + \langle Z, X \rangle \;\geq\; \langle C, X \rangle \quad \text{for feasible } X$$

Example: the dual of the MaxCut SDP

write the objective as $\langle C, X \rangle$ (using $X_{ii} = 1$), where $C = \frac{1}{4}L$ and $L = \mathbf{Diag}(W\mathbf{1}) - W$ is the graph Laplacian (W: edge weights; $\mathbf{Diag}(y)$: diagonal matrix with diagonal y)

$$\begin{array}{ll} \text{maximize}_X & \langle C, X \rangle \\ \text{subject to} & X_{ii} = 1, \quad X \succeq 0 \end{array}$$

we maximize, so build an **upper** bound: for $y \in \mathbb{R}^n$ and $Z \succeq 0$,

$$\mathcal{L}(X, y, Z) = \langle C, X \rangle + \sum_i y_i(1 - X_{ii}) + \langle Z, X \rangle \;\geq\; \langle C, X \rangle \quad \text{for feasible } X$$

$\sup_X \mathcal{L}$ is finite only if $C - \mathbf{Diag}(y) + Z = 0$; then it equals $\mathbf{1}^T y$

$$\begin{array}{ll} \text{minimize}_y & \mathbf{1}^T y \\ \text{subject to} & \mathbf{Diag}(y) - C \succeq 0 \end{array}$$

Certify the triangle

for the triangle, $C = \frac{1}{4}(3I - \mathbf{1}\mathbf{1}^T)$

Certify the triangle

for the triangle, $C = \frac{1}{4}(3I - \mathbf{1}\mathbf{1}^T)$

Q: Find y with $\mathbf{Diag}(y) - C \succeq 0$ and $\mathbf{1}^T y = 9/4$.

Certify the triangle

for the triangle, $C = \frac{1}{4}(3I - \mathbf{1}\mathbf{1}^T)$

Q: Find y with $\mathbf{Diag}(y) - C \succeq 0$ and $\mathbf{1}^T y = 9/4$.
try $y = \frac{3}{4}\mathbf{1}$:

$$\mathbf{Diag}(y) - C = \tfrac{3}{4}I - \tfrac{3}{4}I + \tfrac{1}{4}\mathbf{1}\mathbf{1}^T = \tfrac{1}{4}\mathbf{1}\mathbf{1}^T \succeq 0, \qquad \mathbf{1}^T y = \tfrac{9}{4}$$

Certify the triangle

for the triangle, $C = \frac{1}{4}(3I - \mathbf{1}\mathbf{1}^T)$

Q: Find y with $\mathbf{Diag}(y) - C \succeq 0$ and $\mathbf{1}^T y = 9/4$.
try $y = \frac{3}{4}\mathbf{1}$:

$$\mathbf{Diag}(y) - C = \tfrac{3}{4}I - \tfrac{3}{4}I + \tfrac{1}{4}\mathbf{1}\mathbf{1}^T = \tfrac{1}{4}\mathbf{1}\mathbf{1}^T \succeq 0, \qquad \mathbf{1}^T y = \tfrac{9}{4}$$

- weak duality: **no** feasible X, and so no cut, has value above $9/4$
- our X attains $9/4$: zero gap, so both are optimal
- complementary slackness: $\langle Z, X \rangle = \frac{1}{4}\mathbf{1}^T X \mathbf{1} = 0$, since $X\mathbf{1} = 0$

Certify the triangle

for the triangle, $C = \frac{1}{4}(3I - \mathbf{1}\mathbf{1}^T)$

Q: Find y with $\mathbf{Diag}(y) - C \succeq 0$ and $\mathbf{1}^T y = 9/4$.
try $y = \frac{3}{4}\mathbf{1}$:

$$\mathbf{Diag}(y) - C = \tfrac{3}{4}I - \tfrac{3}{4}I + \tfrac{1}{4}\mathbf{1}\mathbf{1}^T = \tfrac{1}{4}\mathbf{1}\mathbf{1}^T \succeq 0, \qquad \mathbf{1}^T y = \tfrac{9}{4}$$

- weak duality: **no** feasible X, and so no cut, has value above $9/4$
- our X attains $9/4$: zero gap, so both are optimal
- complementary slackness: $\langle Z, X \rangle = \frac{1}{4}\mathbf{1}^T X \mathbf{1} = 0$, since $X\mathbf{1} = 0$

a 3×3 certificate you can check by hand

Exit ticket

1. What is preserved when we replace $\|Ax - b\|_2$ by an epigraph variable?
2. Why does ellipsoidal uncertainty lead to a second-order cone constraint?
3. Which step in the MaxCut formulation changes the feasible decisions?
4. Why must a conic multiplier lie in the dual cone?

Exit ticket

1. What is preserved when we replace $\|Ax - b\|_2$ by an epigraph variable?
2. Why does ellipsoidal uncertainty lead to a second-order cone constraint?
3. Which step in the MaxCut formulation changes the feasible decisions?
4. Why must a conic multiplier lie in the dual cone?

when you change a formulation, ask:

what is preserved? what becomes easier to compute? what is certified?

Midterm practice: robust counterpart by conic duality

earlier, LP duality gave the robust counterpart for a polyhedral uncertainty set; repeat that argument with a ball

1. for fixed v, write $\sup\{v^T u : \|u\|_2 \le \rho\}$ as a conic program in u
2. build its Lagrangian so it is an **upper** bound; derive the dual
3. conclude that $\bar{a}^T x + \rho\|P^T x\|_2 \le b$ if and only if

$$\exists\, \tau : \quad \bar{a}^T x + \rho\tau \le b, \qquad \|P^T x\|_2 \le \tau$$

4. where did you use strong duality? Which constraint qualification holds?

stuck? bring your attempt to office hours

Reference: checking the solver comparison

for square-root Lasso, any y satisfying

$$\|y\|_2 \le 1, \qquad \|A^T y\|_\infty \le \gamma$$

gives a lower bound $-b^T y$ on the optimum

- scale each solver's equality multiplier into this dual feasible set
- recompute the original objective at its returned x for an upper bound
- require relative upper–lower gap and lifted feasibility violation $\le 10^{-6}$

reproduce: `python conic-v1-assets/solver_comparison.py`

protocol, versions, data generation, and raw timings: `conic-v1-assets/README.md` and `solver-results.json`

Reference: quadratic epigraphs

for $z \in \mathbb{R}^m$,

$$\|z\|_2^2 \le t \quad \Longleftrightarrow \quad \left\| \begin{pmatrix} 2z \\ t-1 \end{pmatrix} \right\|_2 \le t+1.$$

Q: Why is squaring the inequality legitimate?

Reference: quadratic epigraphs

for $z \in \mathbb{R}^m$,

$$\|z\|_2^2 \le t \quad \Longleftrightarrow \quad \left\| \begin{pmatrix} 2z \\ t-1 \end{pmatrix} \right\|_2 \le t+1.$$

Q: Why is squaring the inequality legitimate?

- the SOC inequality forces $t + 1 \ge 0$
- squaring gives $4\|z\|_2^2 + (t-1)^2 \le (t+1)^2$, hence $\|z\|_2^2 \le t$
- conversely, $t \ge \|z\|_2^2 \ge 0$, so the squared inequality implies the SOC inequality

Reference: quadratic epigraphs

for $z \in \mathbb{R}^m$,

$$\|z\|_2^2 \le t \quad \Longleftrightarrow \quad \left\| \begin{pmatrix} 2z \\ t-1 \end{pmatrix} \right\|_2 \le t+1.$$

Q: Why is squaring the inequality legitimate?

- the SOC inequality forces $t + 1 \ge 0$
- squaring gives $4\|z\|_2^2 + (t-1)^2 \le (t+1)^2$, hence $\|z\|_2^2 \le t$
- conversely, $t \ge \|z\|_2^2 \ge 0$, so the squared inequality implies the SOC inequality

if $Q = R^T R \succeq 0$, use $z = Rx$ to represent $x^T Q x \le t$

Reference: polyhedral uncertainty

let $\mathcal{U} = \{u : Du \leq d\}$ be nonempty and bounded

$$\sup_{u \in \mathcal{U}} v^T u = \min_{\eta \geq 0} \{d^T \eta : D^T \eta = v\}$$

by LP strong duality

Reference: polyhedral uncertainty

let $\mathcal{U} = \{u : Du \leq d\}$ be nonempty and bounded

$$\sup_{u \in \mathcal{U}} v^T u = \min_{\eta \geq 0} \{d^T \eta : D^T \eta = v\}$$

by LP strong duality

the robust constraint $\bar{a}^T x + \sup_{u \in \mathcal{U}} u^T P^T x \leq b$ is equivalent to

$$\exists \eta : \quad \eta \geq 0, \qquad D^T \eta = P^T x, \qquad \bar{a}^T x + d^T \eta \leq b.$$

Reference: polyhedral uncertainty

let $\mathcal{U} = \{u : Du \leq d\}$ be nonempty and bounded

$$\sup_{u \in \mathcal{U}} v^T u = \min_{\eta \geq 0} \{d^T \eta : D^T \eta = v\}$$

by LP strong duality

the robust constraint $\bar{a}^T x + \sup_{u \in \mathcal{U}} u^T P^T x \leq b$ is equivalent to

$$\exists \eta : \quad \eta \geq 0, \qquad D^T \eta = P^T x, \qquad \bar{a}^T x + d^T \eta \leq b.$$

Q: Why is an existential dual variable enough to enforce a constraint for every u?

Reference: polyhedral uncertainty

let $\mathcal{U} = \{u : Du \leq d\}$ be nonempty and bounded

$$\sup_{u \in \mathcal{U}} v^T u = \min_{\eta \geq 0} \{d^T \eta : D^T \eta = v\}$$

by LP strong duality

the robust constraint $\bar{a}^T x + \sup_{u \in \mathcal{U}} u^T P^T x \leq b$ is equivalent to

$$\exists \eta : \quad \eta \geq 0, \qquad D^T \eta = P^T x, \qquad \bar{a}^T x + d^T \eta \leq b.$$

Q: Why is an existential dual variable enough to enforce a constraint for every u?
A: Any dual feasible η gives an upper bound; strong duality makes the best bound exact.

Reference: polyhedral uncertainty

let $\mathcal{U} = \{u : Du \leq d\}$ be nonempty and bounded

$$\sup_{u \in \mathcal{U}} v^T u = \min_{\eta \geq 0} \{d^T \eta : D^T \eta = v\}$$

by LP strong duality

the robust constraint $\bar{a}^T x + \sup_{u \in \mathcal{U}} u^T P^T x \leq b$ is equivalent to

$$\exists \eta : \quad \eta \geq 0, \qquad D^T \eta = P^T x, \qquad \bar{a}^T x + d^T \eta \leq b.$$

Q: Why is an existential dual variable enough to enforce a constraint for every u?
A: Any dual feasible η gives an upper bound; strong duality makes the best bound exact.
each robust row can use its own dual variable

Reference: SOC as a matrix inequality

the **Schur complement** identity, for $A \succ 0$, is

$$\begin{bmatrix} A & B \\ B^T & C \end{bmatrix} \succeq 0 \quad \Longleftrightarrow \quad C - B^T A^{-1} B \succeq 0.$$

Reference: SOC as a matrix inequality

the **Schur complement** identity, for $A \succ 0$, is

$$\begin{bmatrix} A & B \\ B^T & C \end{bmatrix} \succeq 0 \quad \iff \quad C - B^T A^{-1} B \succeq 0.$$

when $t > 0$, take $A = tI$, $B = z$, $C = t$:

$$\begin{bmatrix} tI & z \\ z^T & t \end{bmatrix} \succeq 0 \quad \iff \quad t - \frac{\|z\|_2^2}{t} \geq 0 \quad \iff \quad \|z\|_2 \leq t.$$

Reference: SOC as a matrix inequality

the **Schur complement** identity, for $A \succ 0$, is

$$\begin{bmatrix} A & B \\ B^T & C \end{bmatrix} \succeq 0 \quad \iff \quad C - B^T A^{-1} B \succeq 0.$$

when $t > 0$, take $A = tI$, $B = z$, $C = t$:

$$\begin{bmatrix} tI & z \\ z^T & t \end{bmatrix} \succeq 0 \quad \iff \quad t - \frac{\|z\|_2^2}{t} \geq 0 \quad \iff \quad \|z\|_2 \leq t.$$

Q: What about $t = 0$? What about $t < 0$?

Reference: SOC as a matrix inequality

the **Schur complement** identity, for $A \succ 0$, is

$$\begin{bmatrix} A & B \\ B^T & C \end{bmatrix} \succeq 0 \quad \iff \quad C - B^T A^{-1} B \succeq 0.$$

when $t > 0$, take $A = tI$, $B = z$, $C = t$:

$$\begin{bmatrix} tI & z \\ z^T & t \end{bmatrix} \succeq 0 \quad \iff \quad t - \frac{\|z\|_2^2}{t} \geq 0 \quad \iff \quad \|z\|_2 \leq t.$$

Q: What about $t = 0$? What about $t < 0$?

- $t = 0$: PSD forces $z = 0$ (check each 2×2 principal minor)
- $t < 0$: negative diagonal entries rule out PSD

Reference: matrix completion

the nuclear norm $\|M\|_* = \sum_i \sigma_i(M)$ has the SDP representation

$$\|M\|_* = \min_{U \in \mathbf{S}^m,\, V \in \mathbf{S}^n} \left\{ \frac{\mathbf{tr}(U) + \mathbf{tr}(V)}{2} : \begin{bmatrix} U & M \\ M^T & V \end{bmatrix} \succeq 0 \right\}, \quad M \in \mathbb{R}^{m \times n}.$$

Reference: matrix completion

the nuclear norm $\|M\|_* = \sum_i \sigma_i(M)$ has the SDP representation

$$\|M\|_* = \min_{U \in \mathbf{S}^m,\, V \in \mathbf{S}^n} \left\{ \frac{\mathbf{tr}(U) + \mathbf{tr}(V)}{2} : \begin{bmatrix} U & M \\ M^T & V \end{bmatrix} \succeq 0 \right\}, \quad M \in \mathbb{R}^{m \times n}.$$

fit observed entries $M_{ij} = a_{ij}$, $(i,j) \in \Omega$, while minimizing $\|M\|_*$

- replacing rank minimization by nuclear norm minimization changes the model
- expressing the nuclear norm by this SDP is **exact**

Reference: matrix completion

the nuclear norm $\|M\|_* = \sum_i \sigma_i(M)$ has the SDP representation

$$\|M\|_* = \min_{U \in \mathbf{S}^m,\, V \in \mathbf{S}^n} \left\{ \frac{\mathbf{tr}(U) + \mathbf{tr}(V)}{2} : \begin{bmatrix} U & M \\ M^T & V \end{bmatrix} \succeq 0 \right\}, \quad M \in \mathbb{R}^{m \times n}.$$

fit observed entries $M_{ij} = a_{ij}$, $(i,j) \in \Omega$, while minimizing $\|M\|_*$

- replacing rank minimization by nuclear norm minimization changes the model
- expressing the nuclear norm by this SDP is **exact**

Q: How large is the PSD matrix? Would you use this formulation for a million users?

Reference: matrix completion

the nuclear norm $\|M\|_* = \sum_i \sigma_i(M)$ has the SDP representation

$$\|M\|_* = \min_{U \in \mathbf{S}^m,\, V \in \mathbf{S}^n} \left\{ \frac{\mathbf{tr}(U) + \mathbf{tr}(V)}{2} : \begin{bmatrix} U & M \\ M^T & V \end{bmatrix} \succeq 0 \right\}, \quad M \in \mathbb{R}^{m \times n}.$$

fit observed entries $M_{ij} = a_{ij}$, $(i,j) \in \Omega$, while minimizing $\|M\|_*$

- replacing rank minimization by nuclear norm minimization changes the model
- expressing the nuclear norm by this SDP is **exact**

Q: How large is the PSD matrix? Would you use this formulation for a million users?
the PSD block has order $m + n$; a different algorithm or model may be needed

Further reading and demo

- Boyd and Vandenberghe, **Convex Optimization**, sections 4.3–4.6, 5.9, and 7.5: conic form, generalized inequalities, experiment design
- CVXPY: disciplined convex programming
- Lobo et al.: applications of second-order cone programming
- Ben-Tal, El Ghaoui, and Nemirovski, **Robust Optimization** (2009)
- Goemans and Williamson, MaxCut and satisfiability via semidefinite programming, JACM (1995)

local demo: `python conic-v1-assets/demo.py`

- production plans, protection versus violation probability, infeasibility certificate
- DCP checks, the square-root Lasso solver representation and its dual
- triangle certificate, MaxCut rounding, experiment design, certified gap